

APOSTILA



PROGRAMAÇÃO ORIENTADA A OBJETOS

EM **JAVA**

Conceitos, Princípios e Prática
para Desenvolver Soluções Modernas



Marcos Monteiro Junior
PROFESSOR

Programação Orientada a Objetos em Java

Marcos Monteiro Junior

Conceitos Princípios e Prática

Sumário

1	Introdução a Java	5
1.1	A linguagem Java	5
1.2	Máquina Virtual	5
1.3	HOTSPOT e JIT	6
1.4	Outras nomenclaturas	6
2	Programação em Java	7
2.1	Criando um projeto	7
2.2	Entrada e saída em Java via console	8
2.3	Tipos primitivos de dados em Java	10
2.4	Casting	11
2.4.1	Conversão implícita	11
2.4.2	Conversão explícita	11
2.5	Aritmética: operadores de igualdade e operadores relacionais	12
2.6	Tomada de decisão	13
2.6.1	O comando switch	14
2.7	Arrays	15
2.7.1	Declarando arrays	15
2.8	Estruturas de repetição	17
2.8.1	A estrutura for-each	17
3	Introdução à programação orientada a objetos	22
3.1	Um breve histórico	22
3.2	Por que usar orientação a objetos?	23
3.2.1	Reúso	23
3.2.2	Coesão	23
3.2.3	Acoplamento	24
3.2.4	Gap semântico	24
4	Fundamentos de Orientação a Objetos	27
4.1	Abstração	27
4.2	Objeto	28
4.2.1	Estado de um objeto e ciclo de vida	28
4.2.2	Identidade e igualdade de um objeto	28
4.3	Classe	29
4.4	Encapsulamento	35
4.4.1	Modificadores de acesso	35

Sumário

4.4.2	Getters e Setters	36
4.4.3	Construtores	39
4.4.4	Sobrecarga e assinatura de métodos	39
4.5	Métodos e atributos estáticos	40
4.5.1	Sobre escrita <i>Override</i>	40
4.5.2	Método <code>toString</code>	41
4.5.3	Método <code>equals</code>	41
4.5.4	Atributos estáticos	42
4.6	Representação de classes na UML (Linguagem de Modelagem Unificada) .	43
4.7	Resumo dos principais tópicos abordados	44
5	Relacionamentos entre classes	45
5.1	Herança	45
5.1.1	Tipos de herança	45
5.1.2	Herança simples	46
5.1.3	Herança multiplas	48
5.2	Polimorfismo	55
5.2.1	Final	56
5.3	Associação	57
5.3.1	Associação (Geral)	57
5.3.2	Agregação	58
5.3.3	Composição	59
6	Mecanismos de abstração	68
6.1	Classes abstratas	68
6.1.1	Métodos abstratos	69
6.2	Interfaces	70
6.3	Métodos estáticos e default	71
6.3.1	Herança multipla	71
6.4	Exemplo	71
6.5	Comparação entre classes abstratas e interfaces	75
	Referências Bibliográficas	82

1 Introdução a Java

Iniciaremos a disciplina de Orientação a Objetos abordando a linguagem Java. Escolhemos essa linguagem devido à sua versatilidade, pois ela está presente em uma ampla variedade de sistemas e dispositivos. Além disso, os conceitos de orientação a objetos aprendidos aqui podem ser aplicados em qualquer outra linguagem que suporte esse paradigma; a única diferença será a sintaxe (a forma de escrever o código), que varia conforme as regras de cada uma.

1.1 A linguagem Java

Java é uma linguagem de programação orientada a objetos, criada em um cenário onde desenvolvedores precisavam lidar manualmente com ponteiros e o complexo gerenciamento de memória (comum em linguagens como C ++), além de enfrentar o desafio de sistemas fortemente acoplados a arquiteturas de hardware e sistemas operacionais específicos. A linguagem Java surgiu para resolver esses problemas. Inicialmente projetada pela Sun Microsystems para dispositivos embarcados de baixo processamento como conversores de TV e eletrodomésticos, o objetivo central era a portabilidade sob o lema “Write Once, Run Anywhere” (Escreva uma vez, execute em qualquer lugar). Hoje, o Java ultrapassou suas origens e consolidou-se como uma das linguagens mais versáteis do mercado, sendo amplamente utilizada em diversos contextos, desde sistemas corporativos de grande escala e aplicações web até o desenvolvimento mobile.

1.2 Máquina Virtual

Em linguagens de programação como o C, o código-fonte passa por um rigoroso processo de tradução antes de rodar. Durante essa fase, entram em cena os analisadores léxicos e sintáticos, a construção de árvores sintáticas e as técnicas de recuperação de erros. O objetivo de todas essas etapas é transformar o que foi escrito em código binário (linguagem de máquina), para que possa ser executado diretamente pelo sistema operacional.

Com isso, o binário gerado do programa elaborado é executado somente para o sistema operacional que ele foi compilado. Para a utilização desse programa em outro sistema operacional todo processo de compilação deve ser executado novamente, podendo ter incompatibilidade entre bibliotecas.

Um dos grandes trunfos do Java é a portabilidade, permitindo que o mesmo código seja executado em diferentes sistemas operacionais sem a necessidade de recompilação. Isso é possível porque a linguagem utiliza uma máquina virtual *Java virtual machine* (JVM) que abstrai as particularidades de cada sistema, permitindo que o desenvolvedor foque

na lógica da aplicação sem se preocupar com as especificidades do sistema operacional (S.O.).

O código-fonte (arquivos `.java`) é compilado pelo `javac` em bytecode (arquivos `.class`). Esse bytecode é então interpretado ou compilado em tempo de execução pela JVM. Esta camada atua como um intermediário inteligente: ela pode coletar métricas de performance, gerenciar a alocação de memória e isolar a aplicação do sistema operacional hospedeiro. Esse isolamento é fundamental, pois garante que, se uma JVM for encerrada abruptamente, as demais aplicações do servidor permaneçam inalteradas, protegendo a estabilidade do ambiente como um todo (Cabral (2014)).

Logo um programa feito em Java possui cinco etapas até sua execução completa: editar, compilar, carregar, verificar e executar.

- Editar: etapa onde o programador escreve seu programa.
- Compilar: o `javac` transforma o código da etapa anterior em bytecode (arquivos `.class`).
- Carregar: Os bytecodes são carregados na memória do computador
- Verificar: verificador de bytecode examina seus bytecodes para assegurar que eles são válidos e não violam restrições de segurança do Java
- Executar: A JVM executa os bytecodes fazendo as ações programadas entrarem em funcionamento.

1.3 HOTSPOT e JIT

Na última etapa citada anteriormente, dois componentes da JVM entram em atividade, o HOTSPOT e JIT.

Para garantir um bom desempenho, a JVM utiliza a tecnologia HotSpot. A função dela é monitorar a aplicação e encontrar ‘pontos quentes’ — aquelas partes do código que rodam repetidas vezes, como o interior de um *loop*. É exatamente nessas situações que o Compilador JIT (*Just-In-Time*) é acionado, compilando sob demanda os trechos onde o desempenho poderia cair. Ao compilar em tempo de execução, o JIT consegue avaliar o código rodando e encontrar a melhor árvore sintática para otimizar aquele processo.

1.4 Outras nomenclaturas

Java Runtime Environment (JRE) é o ambiente de execução do Java pela JVM e bibliotecas, usado para executar uma aplicação Java.

Java Development Kit (JDK) é o kit de desenvolvimento Java, ele pode ser integrado a diversas IDEs de programação ou interpretadores de texto.

2 Programação em Java

O desenvolvimento em Java exige a configuração de um ambiente próprio, começando pela instalação do JDK (Kit de Desenvolvimento Java, que contém a JVM), disponível no site da Oracle. Para facilitar o processo, é comum utilizar pacotes completos que já unem o JDK a uma IDE (Integrated Development Environment). Ferramentas clássicas como o Eclipse e o NetBeans continuam sendo ótimas escolhas para esse propósito. No entanto, o mercado atual tem migrado fortemente para editores mais leves, como o Visual Studio Code, ou IDEs modernas, como o IntelliJ IDEA (da empresa JetBrains).

A escolha é de acordo com a necessidade e objetivos que se pretendem atingir. Caso o programador opte por VSCode, é possível encontrar o 'kit' de desenvolvimento completo para download. Nessa apostila será utilizado o VScode, assim como o IntelliJ IDE.

2.1 Criando um projeto

Para criação de um projeto utilizando o IntelliJ, basta clicar em **file->new->project**. Escolher a pasta onde vai se localizar o projeto.

Inicialmente iremos utilizar o próprio IntelliJ como gerenciador do projeto. A Figura 2.1 resume essas etapas.

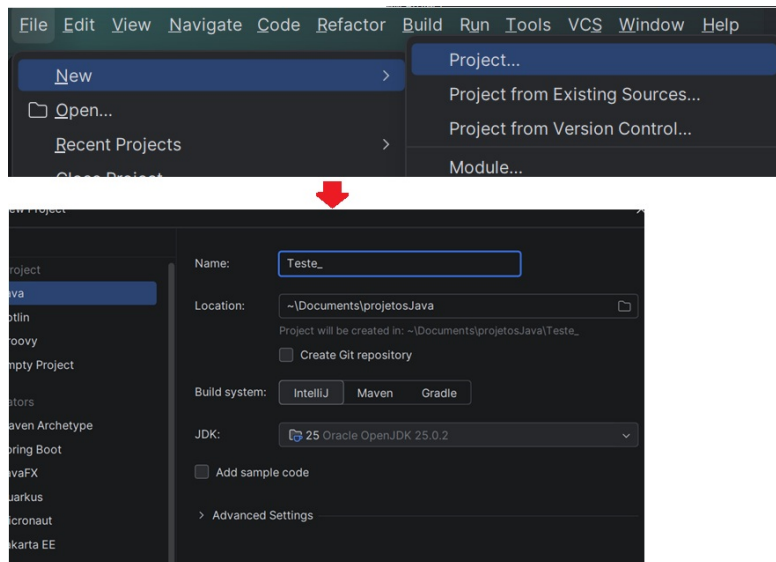


Figura 2.1: Etapas para criação de um Projeto Java.

Depois da criação do projeto, na pasta `src` crie um arquivo do tipo `java.class` de o nome `EntradaSaída`. Na Figura 2.2 está representada essa etapa.

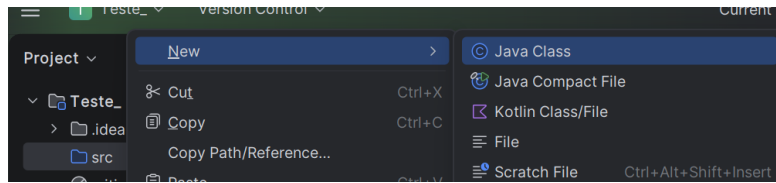


Figura 2.2: Etapas para criação de um arquivo Java class

2.2 Entrada e saída em Java via console

Uma das primeiras etapas no aprendizado de qualquer linguagem de programação é o uso dos recursos de entrada e saída. Em Java, na maioria das versões, é necessário criar uma classe para executar até mesmo o código mais simples. Embora para iniciantes o conceito de classe pareça apenas uma burocracia para garantir a integridade e o formalismo, essa exigência existe porque o Java foi concebido para ser uma linguagem estritamente orientada a objetos. O conceito de classe será discutido nos próximos capítulos.

O arquivo criado na seção anterior deve estar de acordo com a Figura

```
1  import java.util.Scanner;
2
3  public class EntraSaida { no usages
4      |
5  }
```

Figura 2.3: Resultado de um arquivo Java class.

Vamos complementar esse arquivo com alguns códigos. O primeiro código a ser adicionado é: `public static void main(String[] args){ ... }`. Esse trecho de código representa um **método estático público** que diz para a JVM que o projeto deve ser iniciado por esse arquivo.

Abaixo está a explicação de cada uma das palavras reservadas usadas nesse código.

- **public** (Público): É um modificador de acesso. Ele indica que este método é público e pode ser acessado de qualquer lugar.
- **static** (Estático): Indica que este método pertence à classe, e não a um objeto (uma instância da classe).
- **void** (Vazio): É o tipo de retorno.

2 Programação em Java

- `main` (Principal): É o nome do método. Não é uma palavra reservada da linguagem, mas é o nome padronizado e obrigatório que a JVM foi programada para procurar.
- `String[] args` (Argumentos): São os parâmetros que o método recebe.
- `String[]` indica que é um vetor (array) de textos.
- `args` é apenas o nome da variável. Se você executar o programa pelo terminal passando informações extras (ex: `java MeuPrograma arquivo.txt`), esse "arquivo.txt" entrará no programa através dessa variável.

Agora vamos adicionar uma saída de texto no console. Para isso, utilizamos o objeto `System.out`. Os três métodos mais comuns são:

- `System.out.print()`: Imprime o texto no console, mas mantém o cursor na mesma linha.
- `System.out.println()`: Imprime o texto e, ao final, quebra a linha (move o cursor para a linha de baixo). É o mais utilizado.
- `System.out.printf()`: Usado para impressões formatadas (similar à linguagem C), onde você usa marcadores como `%s` para textos e `%d` para números inteiros.

Adicione a mensagem clássica de “Olá mundo” no método `System.out`, ficando assim: `System.out.println("Olá mundo");`. O código final deve ficar como na Figura 2.4

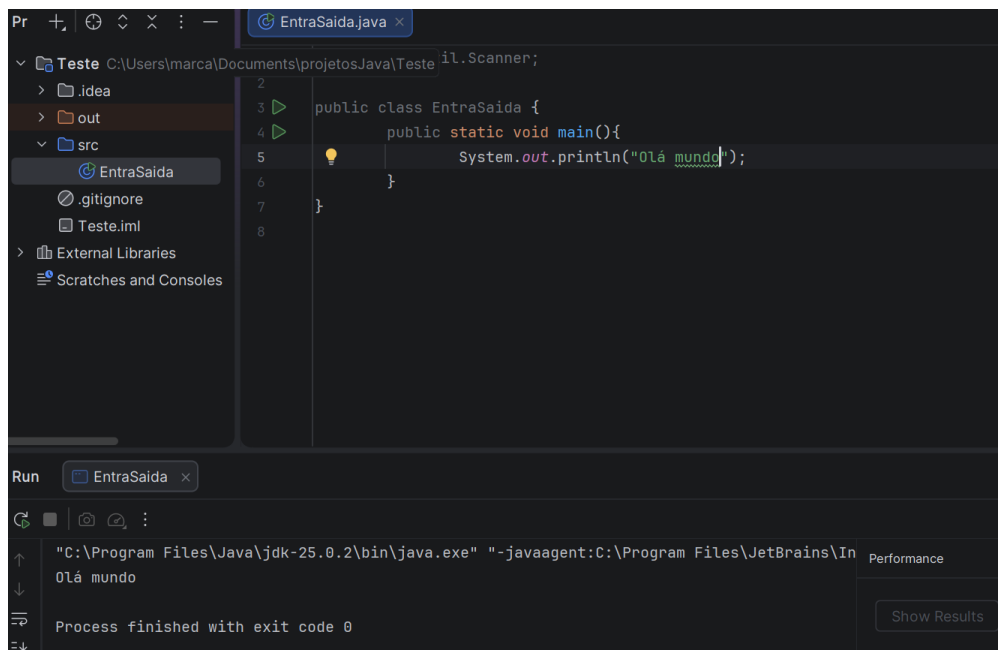


Figura 2.4: Primeira aplicação em Java.

Por outro lado, a entrada de dados em Java via console é feita utilizando a classe `Scanner`. Devemos adicionar um import dessa classe - `import java.util.Scanner;`. Precisamos criar um objeto (discutiremos mais adiante como isso funciona).

A classe `Scanner` tem vários métodos, sendo os mais comuns:

- `nextLine()`: Lê um texto (uma linha inteira) até o usuário apertar Enter.
- `nextInt()`: Lê um número inteiro.
- `nextDouble()`: Lê um número decimal.

Veja o exemplo abaixo:

```
import java.util.Scanner; // 1. Importacao necessaria
public class EntradaSaida {
    public static void main(String[] args) {
        // 2. Instanciando o Scanner
        Scanner teclado = new Scanner(System.in);
        System.out.println("Qual e o seu nome?");
        String nome = teclado.nextLine(); // Le a linha digitada
        // --- SAIDA E ENTRADA DE NUMERO INTEIRO ---
        System.out.println("Quantos anos voce tem?");
        int idade = teclado.nextInt(); // Le o numero digitado
        // --- SAIDA E ENTRADA DE NUMERO DECIMAL ---
        System.out.println("Qual e a sua altura?");
        double altura = teclado.nextDouble(); // Le o decimal
            digitado
        // --- SAIDA FINAL FORMATADA ---
        System.out.println("\n--- Resumo dos Dados ---");
        System.out.println("Nome: " + nome);
        System.out.println("Idade: " + idade + " anos");
        System.out.println("Altura: " + altura + "m");
        // 3. Boa pratica: fechar o Scanner ao final do uso
        teclado.close();
    }
}
```

2.3 Tipos primitivos de dados em Java

Em Java, os tipos primitivos são os “blocos de construção” mais básicos da linguagem. Eles representam valores simples (como números e caracteres) e não são objetos. Por isso, são armazenados diretamente na memória Stack, o que os torna extremamente rápidos e eficientes.

Java possui 8 tipos primitivos, divididos basicamente em quatro categorias:

- Números Inteiros: `int` armazena números inteiros. Esse tipo possui variações do tipo `double`, `short`, `long`, entre outras. Essas variações determinam o tamanho de espaço de memória que será utilizado.

- Ponto flutuante: -float armazena números com valores flutuantes.
- Caractere: -char armazena um caractere.
- Lógica: -boolean é uma variável de valor verdadeiro ou falso.

A declaração dessas variáveis em Java é feita de forma similar a declaração de variáveis em C. Alguns exemplos abaixo:

- `int numero;`
- `float numero_float;`
- `boolean variavel;`

2.4 Casting

O casting em Java é o processo de converter um valor de um tipo de dado para outro. Como o Java é uma linguagem fortemente tipada, ele não permite que você atribua valores de tipos incompatíveis diretamente, a menos que você realize uma conversão explícita ou que a conversão seja segura (“automática”).

Podemos dividir o casting em dois tipos principais: Conversão Implícita (*Widening*) e Conversão Explícita (*Narrowing*)

2.4.1 Conversão implícita

Ocorre quando você converte um tipo de “menor capacidade” para um de “maior capacidade”. Como não há risco de perder dados (o destino comporta o valor original com folga), o Java faz isso automaticamente.

byte → *short* → *int* → *long* → *float* → *double* O caminho inverso não é permitido.

2.4.2 Conversão explícita

Ocorre quando você quer converter de um tipo de “maior capacidade” para um de “menor capacidade”. Aqui há risco de perda de dados (truncamento), por isso o Java exige que você especifique o casting manualmente, usando parênteses antes do valor.

```
double meuDouble = 9.78;
int meuInt = (int) meuDouble; // Explícito: perderemos o .78
```

Atenção Estouro de Memória (*Overflow*): Se você tentar converter um número muito grande (`int` com valor 500) para um tipo pequeno (`byte`, que só vai até 127), o resultado será um valor “estranho” devido ao estouro de bits. É um excelente exemplo prático para mostrar aos alunos por que o casting explícito é perigoso.

2.5 Aritmética: operadores de igualdade e operadores relacionais

A maioria dos programas realiza cálculos aritméticos. Os operadores aritméticos são resumidos na Figura 2.5.

Operação Java	Operador	Expressão algébrica	Expressão Java
Adição	+	$f + 7$	<code>f + 7</code>
Subtração	-	$p - c$	<code>p - c</code>
Multiplicação	*	bm	<code>b * m</code>
Divisão	/	x/y ou $\frac{x}{y}$ ou $x \div y$	<code>x / y</code>
Resto	%	$r \bmod s$	<code>r % s</code>

Figura 2.5: Tabela de operadores aritméticos.

Fonte: Deitel and Deitel (2015)

O Java possui Regras de precedência para operadores semelhante a C. A Figura 2.6 determina a ordem de precedência dos operadores em Java.

Operador(es)	Operação(ões)	Ordem de avaliação (precedência)
* / %	Multiplicação Divisão Resto	Avaliado primeiro. Se houver vários operadores desse tipo, eles são avaliados da <i>esquerda para a direita</i> .
+ - =	Adição Subtração Atribuição	Avaliado em seguida. Se houver vários operadores desse tipo, eles são avaliados da <i>esquerda para a direita</i> . Avaliado por último.

Figura 2.6: Tabela de precedência de operadores.

Fonte: Deitel and Deitel (2015)

Os operadores relacionais e de igualdade são utilizadas para comparar variáveis e condições. A Figura 2.7 mostra os operadores disponíveis em Java.

Operador algébrico	Operador de igualdade ou relacional Java	Exemplo de condição em Java	Significado da condição em Java
<i>Operadores de igualdade</i>			
=	==	<code>x == y</code>	x é igual a y
≠	!=	<code>x != y</code>	x é não igual a y
<i>Operadores relacionais</i>			
>	>	<code>x > y</code>	x é maior que y
<	<	<code>x < y</code>	x é menor que y
≥	>=	<code>x >= y</code>	x é maior que ou igual a y
≤	<=	<code>x <= y</code>	x é menor que ou igual a y

Figura 2.7: Tabela de operadores.

Fonte: Deitel and Deitel (2015)

Operadores de atribuição compostos abreviam expressões de atribuição. Por exemplo, o operador `+=` adiciona o valor da expressão na sua direita ao valor da variável na sua esquerda e armazena o resultado na variável no lado esquerdo do operador. A Figura 2.8 resume as possíveis atribuições compostas.

Operador de atribuição	Expressão de exemplo	Explicação	Atribuições
<i>Suponha:</i> <code>int c = 3, d = 5, e = 4, f = 6, g = 12;</code>			
<code>+=</code>	<code>c += 7</code>	<code>c = c + 7</code>	10 a c
<code>-=</code>	<code>d -= 4</code>	<code>d = d - 4</code>	1 a d
<code>*=</code>	<code>e *= 5</code>	<code>e = e * 5</code>	20 a e
<code>/=</code>	<code>f /= 3</code>	<code>f = f / 3</code>	2 a f
<code>%=</code>	<code>g %= 9</code>	<code>g = g % 9</code>	3 a g

Figura 2.8: Tabela de operadores compostos.

Fonte: Deitel and Deitel (2015)

Java oferece dois operadores unários de incremento e decremento, `++` e `--` respectivamente. Eles podem ser utilizados com pré e pós dependendo se vem antes ou depois da variável. A Figura 2.9 resume as possibilidades de utilização dos operadores unários.

Operador	Nome do operador	Exemplo	Explicação
<code>++</code>	pré-incremento	<code>++a</code>	Incrementa a por 1, então utiliza o novo valor de a na expressão em que a reside.
<code>++</code>	pós-incremento	<code>a++</code>	Usa o valor atual de a na expressão em que a reside, então incrementa a por 1.
<code>--</code>	pré-decremento	<code>--b</code>	Decrementa b por 1, então utiliza o novo valor de b na expressão em que b reside.
<code>--</code>	pós-decremento	<code>b--</code>	Usa o valor atual de b na expressão em que b reside, então decrementa b por 1.

Figura 2.9: Tabela de operadores unários.

Fonte: Deitel and Deitel (2015)

2.6 Tomada de decisão

Uma condição é uma expressão que pode ser **true** ou **false**. O exemplo de instrução de seleção `if` do Java que permite a um programa tomar uma decisão com base no valor de uma condição. Se a condição em uma estrutura `if` for verdadeira, o corpo da estrutura `if` é executado. Se a condição for falsa, o corpo não é executado (Deitel and Deitel (2015)).

Pode-se ainda, acrescentar a execução de comandos quando a estrutura `if` for falsa com o comando **else**. Por fim, é possível aninhar comando `if` e **else**, fazendo uma cadeia de comando condicionais.

Modifique o projeto anterior para executar o exemplo abaixo.

```
import java.util.Scanner;
public class EntradaSaida {
    public static void main(String[] args) {
```

```
Scanner teclado = new Scanner(System.in);
int number1; //declarando as variaveis
int number2;

System.out.println("Digite o primeiro numero");
number1 = teclado.nextInt();
System.out.println("Digite o segundo numero");
number2 = teclado.nextInt();

//Comando if e comparacao de igualdade
if(number1 == number2){
    System.out.println("Os numeros sao iguais");
}else{
    System.out.println("Os numeros sao diferentes");
}
if(number1 < number2){
    System.out.println("Os numero 1 e maior que o
        numero 2");
} else{
    System.out.println("Os numero 2 e maior que o
        numero 1");
}
//Esse if sera executado independente do resultados dos
    anteriores
if (number1 >= number2)
    System.out.printf("numero 1 e maior igual ao numero 2
        %d >= %d", number1, number2);
teclado.close();
}
}
```

2.6.1 O comando switch

O **switch** em Java é uma estrutura de controle de fluxo utilizada para selecionar um bloco de código entre vários possíveis, com base no valor de uma variável. Ele é frequentemente usado como uma alternativa mais limpa ao **if-else-if** encadeado, especialmente quando você está comparando uma variável contra múltiplos valores constantes.

Veja o exemplo de switch para Java 14+ e sua sintaxe:

```
int dia = 3;
String nomeDoDia = switch (dia) {
    case 1 -> "Segunda-feira";
    case 2 -> "Terca-feira";
    case 3 -> "Quarta-feira";
    default -> "Dia invalido";
};
System.out.println(nomeDoDia);
```

2.7 Arrays

Um array é um grupo de variáveis (chamados elementos ou componentes) que contém valores todos do mesmo tipo. Os arrays são objetos; portanto, são considerados tipos por referência Deitel and Deitel (2015).

2.7.1 Declarando arrays

Assim como outros objetos, os arrays ocupam espaço na memória e são instanciados por meio da palavra-chave `new`. Para criar um array, é necessário especificar o tipo dos seus elementos e a quantidade desejada, definindo essas informações diretamente na expressão de criação (Deitel and Deitel (2015)). Isso tudo porque podemos trabalhar com arrays de qualquer tipo de dado, desde tipos primitivos até objetos complexos, e é nos objetos complexos que a declaração de Arrays dessa forma se mostram úteis.

```
int[] c; // declara a variavel de array
c = new int[12]; // cria o array; atribui a variavel de array
```

Veja o exemplo abaixo que aborda os conceitos de Programação Básica em Java vistos nessa unidade. O exemplo consiste em um programa que solicita a nota de um aluno, calcula a média e determina se o aluno foi aprovado ou reprovado. Além disso, o programa percorre as notas mostrando quais foram as notas acima da sua própria média.

```
import java.util.Scanner;
public class Exemplo {
    public static void main(String[] args) {
        Scanner entrada = new Scanner(System.in);
        System.out.println("=== BEM-VINDO AO SISTEMA DE NOTAS E FALTAS ===");
        System.out.print("\nQuantas notas o aluno possui? ");
        int quantidadeNotas = entrada.nextInt();
        if (quantidadeNotas <= 0) {
            System.out.println("Erro: A quantidade de notas deve ser maior que zero!");
            return;
        }
        double[] notas = new double[quantidadeNotas];
        double somaNotas = 0.0;
        System.out.println("\n--- Cadastro de Notas ---");
        for (int i = 0; i < notas.length; i++) {
            System.out.printf("Digite a nota %d: ", (i + 1));
            notas[i] = entrada.nextDouble();
            entrada.nextLine();
            somaNotas = somaNotas + notas[i];
        }
        double media = somaNotas / quantidadeNotas;
        System.out.printf("\nA média final do aluno e: %.2f\n", media);
        System.out.println("\n--- Status de Aprovação ---");
```

```

boolean aprovadoPorNota;
if (media >= 7.0) {
    System.out.println("Desempenho: Aluno APROVADO por média.");
    aprovadoPorNota = true;
} else if (media < 7.0 && media >= 2.0) {
    System.out.println("Desempenho: Aluno em EXAME FINAL.");
    aprovadoPorNota = false;
} else {
    System.out.println("Desempenho: Aluno REPROVADO DIRETO por nota.");
    aprovadoPorNota = false;
}
System.out.println("\n--- Avaliação Qualitativa da média ---");
// Convertendo a média para inteiro apenas para demonstrar o uso classico do switch
int mediaInteira = (int) media;
String avaliacao = switch (mediaInteira) {
    case 10, 9 -> "Excelente (Desempenho excepcional!);
    case 8, 7 -> "Bom (Bom trabalho!);
    case 6, 5 -> "Regular (Atingiu o minimo esperado.);
    case 4, 3 -> "Ruim (Precisa estudar muito mais.);
    case 2, 1, 0 -> "Critico (Desempenho insatisfatorio.);
    ;
    default -> {
        // Caso precise de lógica extra em um case, usamos blocos com 'yield'
        yield "média fora da escala esperada (0 a 10).";
    }
};
System.out.println("\n--- Histórico de Notas Lançadas ---");
System.out.println("Verificando notas individuais acima da média da turma...");
for (double notaAtual : notas) {
    // Usando operador de igualdade (==) e relacional (>)
    if (notaAtual == 10.0) {
        System.out.println("-> Nota " + notaAtual + " (Nota Máxima - Parabens!);
    } else if (notaAtual > media) {
        System.out.println("-> Nota " + notaAtual + " (Acima da média geral);
    } else {
        System.out.println("-> Nota " + notaAtual + " (Abaixo ou na média geral);
    }
}

```

```

        // Boa pratica: Fechar o scanner para liberar recursos
        entrada.close();
        System.out.println("\n=== PROGRAMA ENCERRADO ===");
    }
}

```

2.8 Estruturas de repetição

Em Java, os laços de repetição (loops) são fundamentais para automatizar tarefas repetitivas. Tanto o `for` quanto o `while` servem para repetir um bloco de código, mas a escolha entre eles depende de como você controla a repetição.

O `for` é geralmente utilizado quando você sabe exatamente quantas vezes o código deve repetir. Ele condensa a inicialização, a condição e a atualização em uma única linha. Veja a sintaxe abaixo, muito semelhante a C.

```

for(inicializacao; condicao; atualizacao){
    //comandos;
}
while(condicao){
    //comandos;
    //comando para finalizacao;
}

```

2.8.1 A estrutura for-each

O `for-each` (oficialmente chamado em Java de *Enhanced For Loop*) é uma das ferramentas mais utilizadas na linguagem. Ele foi introduzido no Java 5 com um objetivo principal: simplificar a leitura de coleções e **arrays**. Em vez de se preocupar com índices, o `for-each` foca no elemento em si. Funciona para cada elemento dentro da coleção.

Veja o exemplo abaixo, a saída serão os nomes em ordem da lista:

```

...
String[] alunos = {"Ana", "Bia", "Caio"};
for (String nome : alunos) {
    System.out.println(nome);
}

```

A Tabela 2.1 pode ajudar na decisão de qual estrutura utilizar entre `for` e `for-each`

Cenário	Melhor escolha	Motivo
Percorrer toda a lista	<code>for-each</code>	É mais legível e menos propenso a erros de índice.
Precisar do índice (i)	<code>for</code> comum	O <code>for-each</code> não te dá acesso à posição (ex: $i = 0$).
Modificar a estrutura	<code>for</code> comum	Tentar remover itens de uma lista usando <code>for-each</code> causa erro.
Percorrer ordem inversa	<code>for</code> comum	O <code>for-each</code> sempre percorre do primeiro ao último.

Tabela 2.1: Comparativo: `for` tradicional vs. `for-each`.

Exercícios

Utilizando uma classe e o método *main* resolva os exercícios abaixo sobre tipos primitivos e estruturas básicas de Java;

1. Escreva instruções para realizar cada uma das tarefas a seguir:
 - a) Declare que as variáveis *c*, *istoEVariavel*, *q1031* e *numero* serão do tipo `int`.
 - b) Solicite para o usuário inserir um valor inteiro.
 - c) Insira um inteiro e atribua o resultado à variável `int` *valor*. Suponha que a variável `Scanner` *input* possa ser utilizada para ler.
 - d) Imprima no console: “Este e um programa em Java”. Utilize o método `System.out.println`.
 - e) Se a variável *numero* não for igual a 7, exiba “Esta variavel nao e igual a 7”;
2. Identifique e corrija os erros em cada uma das seguintes instruções:
 - a) `if(c < 7);`
`System.out.println(“c e menor que 7”);`
 - b) `if(c => 7)`
`System.out.println(“c e igual ou maior que 7”);`
3. Crie uma calculadora simples que realiza operações aritméticas básicas entre dois números inteiros fornecidos pelo usuário. ,
 - a) Crie a classe principal: Declare uma classe chamada “CalculadoraSimples” contendo o método *main*.
 - b) Importe e configure a leitura: Importe a classe `Scanner` e instancie um objeto para permitir a leitura de dados via console.
 - c) Declaração de variáveis: Crie três variáveis do tipo `int` (chamadas, por exemplo, *numero1*, *numero2* e *operacao*) para armazenar os valores que serão digitados pelo usuário.
 - d) Entrada de dados: Imprima na tela uma mensagem solicitando que o usuário digite o primeiro número e armazene-o na variável *numero1*. Repita o processo para o segundo número e armazene-o em *numero2*.
 - e) Solicite que o usuário digite o número correspondente à operação desejada. Exemplo de menu de opções: “Digite um número para a operação:”
 - 1 - Somar
 - 2 - Subtrair
 - 3 - Multiplicar
 - 4 - Dividir

- f) Seleção via Switch: Utilize a estrutura `switch` para verificar o valor da variável opção e executar apenas a operação selecionada.
 - g) Processamento e Saída: Dentro de cada case, realize o cálculo correspondente e imprima o resultado na tela.
 - h) Exibição dos resultados: Utilize `System.out.println` (ou `printf`) para imprimir na tela o resultado de cada operação com uma frase explicativa.
 - Exemplo: “A soma de X e Y é: Z”.
 - i) Tratamento de exceção: Adicione um bloco de verificação (`if`) para evitar a divisão por zero. Se o segundo número for zero, imprima uma mensagem de erro (“Divisão por zero não permitida”) em vez de tentar calcular a divisão.
 - j) Finalização: Lembre-se de encerrar o objeto Scanner usando o método `.close()` ao final do seu código.
4. Dado que $y = ax^3 + 7$, quais das seguintes alternativas são instruções Java corretas para essa equação?
- a) $y = a * x * x * x + 7;$
 - b) $y = a * x * x * (x + 7);$
 - c) $y = (a * x) * x * (x + 7);$
 - d) $y = (a * x) * x * x + 7;$
 - e) $y = a * (x * x * x) + 7;$
 - f) $y = a * x * (x * x + 7);$
5. Faça um aplicativo Java que calcula e imprime a soma dos inteiros de 1 a 10. Utilize a instrução `while` para fazer `loop` pelas instruções de cálculo e incremento. O `loop` deve terminar quando o valor de x tornar-se 11.
6. (**Tabela de quadrados e cubos**) Utilizando apenas as técnicas de programação que aprendeu neste capítulo, escreva um aplicativo que calcule os quadrados e cubos dos números de 0 a 10 e imprima os valores resultantes em formato de tabela como a seguir:

Número	Quadrado	Cubo
0	0	0
1	1	1
2	4	8
3	9	27
4	16	64
5	25	125
6	36	216
7	49	343
8	64	512
9	81	729
10	100	1000

Tabela 2.2: Tabela de quadrados e cubos de 0 a 10.

7. (**Calculadora de economia da carona solidária**) Crie um aplicativo que calcule o custo diário de dirigir, para estimar quanto dinheiro pode ser economizado com o uso da carona solidária, que também tem outras vantagens, como reduzir emissões de carbono e congestionamento de tráfego. O aplicativo deve introduzir as seguintes informações e exibir o custo por dia de dirigir para o trabalho do usuário:
- Quilômetros totais dirigidos por dia.
 - Preço por litro de gasolina.
 - Quilômetros médios por litro.
 - Taxas de estacionamento por dia.
 - Pedágio por dia

8. Identifique e corrija os erros em cada um dos seguintes conjuntos de código:

a)

```
while (c <= 5){
    product *= c;
    ++c;
```

b)

```
if (gender == 1)
    System.out.println("Woman");
else;
    System.out.println("Man");
```

3 Introdução à programação orientada a objetos

Toda a base da programação deriva dos algoritmos. Estes consistem em sequências finitas de passos bem definidos, com início e fim claramente delimitados, que buscam a automação de processos. Um algoritmo deve ser suficientemente genérico para que sua implementação seja viável em diferentes linguagens de programação.

A característica fundamental de uma linguagem, além de sua sintaxe, é o seu **paradigma** de programação. Este define a abordagem sob a qual o algoritmo deve ser estruturado para que seus passos alcancem os objetivos propostos Leite et al. (2016).

O paradigma mais utilizado para introduzir a lógica de programação é o **estruturado/procedural**. Por ser um dos modelos mais simples e intuitivos para a expressão de algoritmos, ele frequentemente permite uma transição direta entre a representação lógica e a implementação na linguagem. As linguagens mais conhecidas que adotam esse paradigma são C e Pascal Leite et al. (2016).

Com o avanço do tempo e a crescente demanda por sistemas complexos, o paradigma estruturado tornou-se insuficiente para as novas exigências da engenharia de software. A necessidade de abstrações que aproximassem a tecnologia das entidades do cotidiano impulsionou a criação e o desenvolvimento do Paradigma Orientado a Objetos.

3.1 Um breve histórico

A origem da programação e do paradigma orientado a objetos vem da Universidade de Oslo na Noruega na década de 60. Os pesquisadores Kristen e Ole desenvolveram a primeira linguagem orientada a objetos a “Simula” com objetivo de simulações de eventos discretos (daí seu nome).

Apesar da Simula ser a pioneira, ela foi desenvolvida para uma máquina (UNIVAC 1107) o que a tornava pouco portátil. entretanto na década de 70, Alan Kay desenvolveu uma linguagem orientada a objeto que se tornaria uma das mais conhecidas, a Smalltalk-80.

Muitas linguagens surgiram depois de Smalltalk-80, como C++, Java, C#, Python e etc. E apesar de não ser muito utilizada nos dias atuais, a Smalltalk serviu de inspiração para muitas linguagens orientadas a objetos (O.O).

3.2 Por que usar orientação a objetos?

Além da programação estruturada não representar as informações próximo ao mundo real, ela foca em dados e operações desassociadas. Ou seja, diversos conceitos ficam misturados, logo, não fica claro qual operação está ligada a um determinado dado Leite et al. (2016).

A orientação a objetos foca na organização e interação dos objetos entre si. A Figura 3.1 mostra a diferença entre os paradigmas.

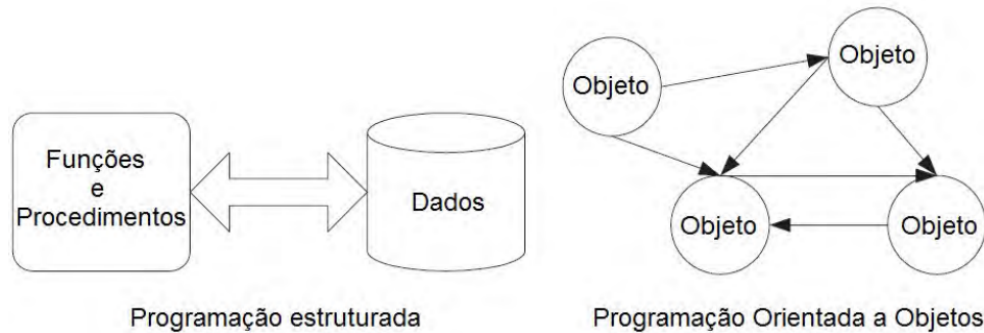


Figura 3.1: Comparativos entre paradigmas de programação.

Fonte: Leite et al. (2016)

Outros fatores para o uso de O.O. descritos por Leite et al. (2016) estão nas subseções a seguir.

3.2.1 Reúso

O reaproveitamento em linguagens estruturadas é ineficiente e propenso a falhas, muitas vezes dependente de cópias manuais ou de sistemas de módulos pouco integrados. A Orientação a Objetos supera isso ao utilizar mecanismos como a herança e a associação, que permitem estabelecer conexões estruturais entre componentes, tornando o reuso prático, seguro e mais próximo das necessidades reais do desenvolvimento de software (Leite et al. (2016)).

3.2.2 Coesão

A organização interna do sistema é um ponto crítico. Em modelos estruturados, é comum que funções ou módulos acumulem múltiplas responsabilidades, criando blocos de código densos e difíceis de manter.

O paradigma orientado a objetos propõe a criação de classes que concentram cada tarefa em uma unidade específica e bem definida. Esse foco em uma única responsabilidade facilita a leitura, a manutenção e a expansão do sistema sem gerar efeitos colaterais indesejados (Leite et al. (2016)).

3.2.3 Acoplamento

O excesso de dependências entre diferentes partes de um software gera fragilidade, onde alterações em um ponto podem interromper funcionalidades em outro.

A Orientação a Objetos introduz estruturas que possibilitam um acoplamento mais flexível, permitindo que as classes interajam de forma controlada. Isso resulta em sistemas mais independentes, nos quais os componentes podem evoluir sem comprometer a integridade de todo o conjunto (Leite et al. (2016)).

3.2.4 Gap semântico

Programar em modelos estruturados exige que o desenvolvedor foque excessivamente na lógica de execução técnica da máquina, distanciando o código dos conceitos do cotidiano.

A Orientação a Objetos atua como uma ponte, oferecendo ferramentas para que a estrutura do programa reflita diretamente os termos, os processos e a lógica do domínio do problema. Essa aproximação reduz o esforço mental necessário para traduzir desafios reais em soluções computacionais eficientes (Leite et al. (2016)).

A tabela 3.1 apresenta um comparativo entre os paradigmas estruturado e orientado a objetos.

Tabela 3.1: Comparativo Avançado: Programação Estruturada vs. Orientação a Objetos

Critério	Programação Estruturada	Orientação a Objetos
Foco Principal	Focado em funções, procedimentos e sequências de passos lógicos.	Focado na organização e interação direta entre objetos.
Visão de Dados	Dados e operações encontram-se separados e desassociados.	Dados (atributos) e comportamentos (métodos) encapsulados em uma única unidade (classe).
Reúso	Ineficiente e manual; propenso a falhas.	Prático e seguro via herança e associação.
Coesão	Módulos e funções acumulam múltiplas responsabilidades.	Classes focadas em uma única responsabilidade.
Acoplamento	Excesso de dependências rígidas e frágeis entre as partes.	Flexível e controlado, garantindo independência aos componentes.
Gap Semântico	Alto; focado excessivamente na lógica técnica da máquina.	Baixo; a estrutura do código reflete o domínio do problema.
Extensibilidade	Mais complexa e custosa em grandes sistemas devido ao acoplamento.	Alta, permitindo estender funcionalidades de forma isolada via herança e polimorfismo.

Para demonstrar as diferenças entre os paradgmas, vamos utilizar um exemplo classico

de cálculo de área de figuras geométricas. A Figura 3.2 abaixo mostra dois códigos, sendo um procedural em C e outro orientado a objetos em Java.

Procedural (Linguagem C)	Orientada a Objetos (Java)
<pre>#include <stdio.h> struct Retangulo { double largura; double altura; }; double calcularArea(struct Retangulo r) { return r.largura * r.altura; } int main() { struct Retangulo r; r.largura = 10.0; r.altura = 5.0; double a = calcularArea(r); printf("Area: %.2f", a); return 0; }</pre>	<pre>public class Retangulo { private double largura; private double altura; public Retangulo(double l, double a) { this.largura = l; this.altura = a; } public double calcularArea() { return this.largura * this.altura; } } // Na Principal: Retangulo r = new Retangulo(10, 5); double a = r.calcularArea();</pre>

Figura 3.2: Comparação estrutural entre Programação Procedural (C) e POO (Java).

Na comparação dos dois códigos podemos observar a separabilidade vs. união. No código procedural (C), os dados (`struct Retangulo`) e as ações (`calcularArea`) são totalmente separados. Os dados são passivos e a função atua de fora, recebendo a estrutura como parâmetro. No código orientado a objetos (Java), dados e comportamento são unidos dentro da mesma classe, e o objeto sabe calcular sua própria área internamente através do contexto (`this`).

Outra característica observada na comparação entre os códigos é a chamada e o acoplamento. Na abordagem procedural, você precisa gerenciar manualmente a criação da estrutura, preencher seus campos e passá-la explicitamente para uma função externa. Na POO, a inicialização ocorre de forma coesa por meio do construtor, e a chamada do método é feita diretamente a partir da instância (`r.calcularArea()`), garantindo maior organização e encapsulamento dos dados.

No próximo capítulo, vamos explorar conceitos iniciais de programação orientada a objetos, incluindo classes, objetos, métodos e encapsulamento. Esses conceitos são fundamentais para criar sistemas em Java. No próximo capítulo serão abordados os primeiros conceitos de orientação a objetos.

Exercícios

1. Definições Conceituais Explique com suas próprias palavras os seguintes conceitos abordados no capítulo:
 - a) O que motivou a criação do paradigma orientado a objetos em substituição ao paradigma estruturado?
 - b) Qual foi o contexto histórico de criação das primeiras linguagens orientadas a objetos (como Simula e Smalltalk-80)?
2. Comparação de Paradigmas; O texto aponta que a programação estruturada foca em dados e operações desassociadas, enquanto a orientação a objetos foca na organização e interação entre objetos. Com base nisso:
 - a) Explique como a orientação a objetos melhora o reúso de código em comparação aos modelos estruturados.
 - b) Defina o que é coesão e por que criar classes focadas em uma única responsabilidade beneficia a manutenção do sistema.
3. Acoplamento e Gap Semântico
 - a) O que é o acoplamento em um software e por que o excesso de dependências entre diferentes partes de um código torna o sistema frágil?
 - b) Explique o conceito de gap semântico e de que maneira a orientação a objetos atua como uma "ponte" para reduzir o esforço mental do desenvolvedor ao traduzir problemas do mundo real.

4 Fundamentos de Orientação a Objetos

Após a discussão sobre as diferenças entre os paradigmas de programação, este capítulo aprofunda-se nos conceitos fundamentais da Programação Orientada a Objetos (POO), apresentando definições e exemplos práticos. Paralelamente, serão introduzidos os diagramas de classes da (Linguagem de Modelagem Unificada) UML, que servirão de suporte para a modelagem dos sistemas abordados ao longo do material.

4.1 Abstração

O próximo conceito parte de algo do nosso cotidiano. A abstração nada mais é que uma forma de generalizar as características de um objeto. A abstração pode estar ligada diretamente ao que se deseja atingir, logo o problema que se deseja resolver pode influenciar em diferentes níveis de abstração.

Vamos a um exemplo: uma clínica veterinária deseja adicionar ao seu sistema uma forma de cadastrar informações no seu sistema, inicialmente ela deseja registrar os pets. De acordo com a definição de abstração, a empresa deve definir os animais da maneira mais simples possível. Portanto, será possível criar variações de animais como cachorro, gato, passaros entre outros. Observe a Figura 4.1 abaixo.

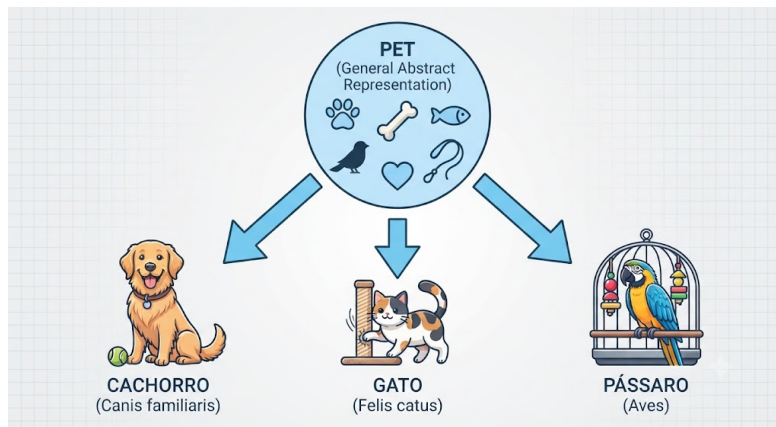


Figura 4.1: Abstração de pets para cadastrar derivados como cachorros, gatos e pássaros.
Fonte: Imagem gerada por IA

Saber abstrair as informações que são importantes para um sistema é algo fundamental para a construção de software. Essa habilidade é adquirida com a prática e com o tempo.

4.2 Objeto

Uma das definições mais importantes é o objeto. Segundo Leite et al. (2016) “*Um objeto é a representação de um conceito/entidade do mundo real, que pode ser física (bola, carro, árvore etc.) ou conceitual (viagem, estoque, compra etc.) e possui um significado bem definido para um determinado software.*”.

No exemplo anterior da clínica veterinária o objeto é o animal, assim como em um sistema de estoque o objeto pode ser o produto. Outro exemplo de objeto é uma viagem, que pode ter como atributos o destino, a data de ida e volta, o preço, entre outros.

Um objeto pode ser claramente definido através de:

- Um conjunto de atributos (características do objeto, o que ele é)
- Um conjunto de métodos (comportamento do objeto, o que ele faz)

4.2.1 Estado de um objeto e ciclo de vida

Depois que um objeto é criado, passa a ter um estado definido pelos valores de seus atributos. Este estado pode mudar ao longo do tempo conforme alterações feitas nos seus atributos pelos seus métodos. Um exemplo é um possível método `vacinar()` na classe `Pet` que altera o estado do atributo `vacinado` de *false* para *true*.

O ciclo de vida de um objeto é definido por três fases:

- Criação
- Uso
- Destruição

A criação ocorre quando o objeto é instanciado a partir de sua **classe**, o uso envolve a manipulação de seus atributos e métodos, e a destruição acontece quando o objeto deixa de ser necessário e é removido da memória pelo coletor de lixo do Java.

Observação

Apesar dos termos *instância* e *objeto* serem usados frequentemente como sinônimos, eles possuem significados distintos. **Objeto** é o conceito genérico de uma entidade criada a partir de uma classe, enquanto **instância** é o objeto concretizado (com valores), olhando especificamente para a relação dele com a sua classe de origem.

4.2.2 Identidade e igualdade de um objeto

A identidade de um objeto é única e imutável. Devido a isso, é muito difícil de comparar dois objetos, mesmo que eles possuam os mesmos atributos e valores. Para isso, o Java fornece o método `equals()` que compara se dois objetos são iguais, ou seja, se possuem os mesmos valores de atributos.

Esse e outros métodos serão tratados na seção (métodos e atributos estáticos) 4.5.

4.3 Classe

Um dos conceitos fundamentais da Programação Orientada a Objetos (POO) é a classe. Ela funciona como um molde ou modelo para a criação de objetos, representando a abstração de um conceito do mundo real. Na classe, definimos as características (**atributos**) e as ações (**métodos**) que os objetos desse tipo deverão possuir. Portanto, enquanto a classe é a definição generalizada, o objeto é a instância específica e concreta gerada a partir desse modelo.

Para nomear uma classe devemos utilizar **substantivos** e como uma boa prática deve iniciar com letra maiúscula. Em Java é comum criar um novo arquivo para cada classe do sistema. **Este arquivo deve ter o mesmo nome da classe.**

Veja o exemplo abaixo de uma classe baseada no exemplo da “pet”. Para replicar esse exemplo, crie um projeto Java como foi feito no capítulo 1 na seção 2.1. Depois de criado o projeto, crie um arquivo novo do tipo *Java Class* na pasta **src**, de o nome de Pet, em seguida insira o código abaixo.

```
public class Pet {
    String nomeAnimal;
    int idadeAnimal;
    int sexoAnimal;
    boolean vacinado;
}
```

No exemplo anterior, definimos que as características pertinentes ao objeto Pet é o nome, a sua idade, seu sexo e o status de vacinação.

Agora na classe Principal onde está o método **main**, vamos criar o objeto e torna-lo concreto. Para isso basta utilizar a palavra chave **new**. Veja o exemplo abaixo:

```
public class Principal {
    public static void main(String[] args) {
        new Pet();
    }
}
```

O objeto é criado na memória, mas para acessá-lo, é necessário atribuí-lo a uma variável. Somente por meio de uma **referência** (uma variável nunca acessa o objeto diretamente na memória, sempre usamos a referência dele) podemos manipular esse objeto sem a necessidade de gerenciar a memória diretamente. Como vimos, o Java é uma linguagem fortemente tipada, o que significa que toda variável deve possuir um tipo definido. Portanto, para armazenar uma **instância** de “Pet”, devemos declarar uma variável do tipo “Pet”.

```
public class Principal {
    public static void main(String[] args) {
        Pet novoPet;
        novoPet = new Pet();
    }
}
```

```
}
```

Agora podemos adicionar valores às características do Pet, para utilizar os recursos (atributos, métodos, etc) de uma classe usamos a variável+“.”+recurso.

```
public class Principal {
    public static void main(String[] args) {
        Pet novoPet;
        novoPet = new Pet();
        novoPet.nomeAnimal = "Wando";
        novoPet.idadeAnimal = 1;
        novoPet.SexoAnimal = 1;
        novoPet.vacinado = false;
        System.out.println("O sexo do pet e: " +novoPet.sexoAnimal)
            ; //o sinal + em uma string significa ‘concatenar’
    }
}
```

Como vimos anteriormente, os métodos definem as ações que um objeto pode realizar ou o comportamento que ele pode exibir. Para ilustrar isso, vamos utilizar o atributo vacinado da classe Pet, esse atributo controlará o histórico de vacinação do animal. Além disso, implementaremos o método vacinar(), que será responsável por alterar esse estado de false para *true*, registrando a vacinação no sistema.

```
public class Pet {
    String nomeAnimal;
    int idadeAnimal;
    int sexoAnimal;
    boolean vacinado;
}
void vacinar() {
    this.vacinado = true;
    System.out.println(this.nomeAnimal + " foi vacinado com
        sucesso!");
}
}
```

O método vacinar() altera o status do objeto Pet para indicar que ele recebeu a dose necessária. Neste método, utilizamos a palavra-chave **this** para referenciar o atributo de instância vacinado da classe, diferenciando-o de qualquer parâmetro ou variável local que pudesse ter o mesmo nome. Essa prática é essencial para garantir que a alteração de estado seja aplicada corretamente ao objeto específico que está sendo vacinado. O arquivo principal deve ser estruturado da seguinte forma:

```
public class Principal {
    public static void main(String[] args) {
        // Instanciação e manipulação do objeto Pet
        Pet novoPet;
        novoPet = new Pet();
    }
}
```

4 Fundamentos de Orientação a Objetos

```
// Simulação de uma ação de vacinação
// O método vacinar altera o estado interno do objeto
novoPet.vacinar();

// Exemplo de acesso e exibição de informações
System.out.println("Status do pet vacinado: " + novoPet.
    vacinado);
}
```

Exercícios**Exercícios**

1. Defina:

- a) Abstração
- b) Classe
- c) Atributo
- d) Método

2. Implemente a classe representada pela Figura 4.2.

- O método “liga” deve apresenta a mensagem “O carro está ligado”.
- O método “acelera” deve receber um valor e esse valor deve somar na variável velocidade.
- O método “pegaMarcha” deve informar a marcha que o carro está de acordo com sua velocidade.
 - caso a velocidade seja menor 0 o método deve retornar -1
 - se a velocidade estiver entre 0 e 39 deve retornar 1
 - caso esteja entre 40 e 79 deve retornar 2
 - por último, se estiver acima de ou igual a 80 deve retornar 3

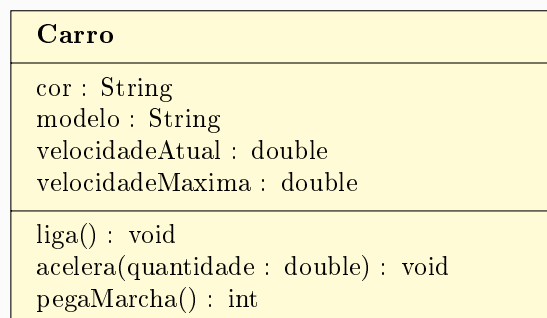


Figura 4.2: Diagrama UML da classe Carro.

O teste deve ser executado na classe principal com os seguintes valores:

```
public class Principal{
    public static void main(String[] args) {
        Carro meuCarro;
        meuCarro = new Carro();
        meuCarro.cor = "Verde";
        meuCarro.modelo = "Fusca";
        meuCarro.velocidadeAtual = 0;
    }
}
```

```

        meuCarro.velocidadeMaxima = 80;
        // liga o carro
        meuCarro.liga();
        // acelera o carro
        meuCarro.acelera(20);
        System.out.println(meuCarro.velocidadeAtual);
    }
}

```

3. Crie uma porta (Figura 4.3), abra e feche a mesma, pinte-a de diversas cores, altere suas dimensões e use o método `estaAberta` para verificar se ela está aberta.

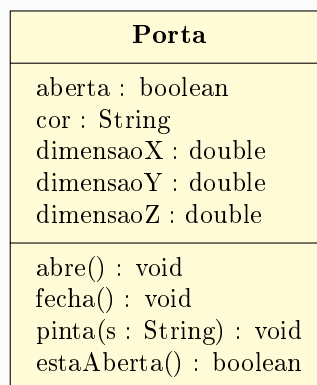


Figura 4.3: Diagrama UML da classe Porta.

Faça uma rotina de testes na classe principal para testar a classe Porta, como por exemplo: Crie uma porta, abra e feche a mesma, pinte-a de diversas cores, altere suas dimensões e use o método `estaAberta` para verificar se ela está aberta.

4. Crie um pacote de viagem (Figura 4.4), altere seu destino, ajuste o valor base, adicione e remova dias de duração, aplique um desconto promocional (sobre o valor de base ex: $\text{valorBase} -= \text{valorBase} * (\text{percentual} / 100.0)$) e use o método `calcularValorTotal` ($\text{valorBase} * \text{duracaoDias}$) para verificar o preço final do pacote.

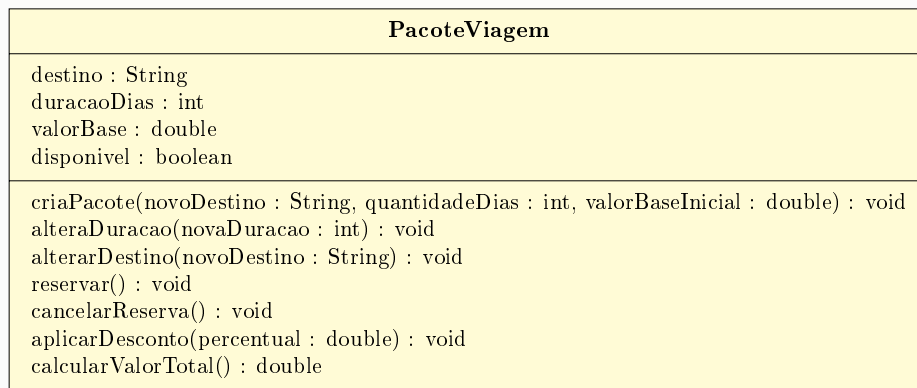


Figura 4.4: Diagrama UML da classe PacoteViagem.

Faça uma rotina de testes na classe principal para testar a classe **PacoteViagem**, como por exemplo: Crie um pacote de viagem para um destino inicial, altere o destino, ajuste o valor base e a duração, realize uma reserva, aplique um desconto promocional e exiba o valor total calculado.

4.4 Encapsulamento

O encapsulamento é um dos conceitos mais utilizados na Programação Orientada a Objetos. A ideia consiste em ocultar informações e a complexidade interna de um objeto, disponibilizando apenas o necessário para quem o utiliza. Dessa forma, é possível blindar as características internas do objeto em relação a outras classes, evitando resultados inesperados, acessos indevidos e outros problemas de integridade. Nessa seção vamos ver algumas formas de fazer encapsulamento.

4.4.1 Modificadores de acesso

Em Java o encapsulamento pode ser feito com os modificadores de acesso. Os modificadores são palavras que dizem ao sistema quais informações podem ser acessadas fora da classe ou por classes que pertencem ao mesmo pacote da classe atual. Os modificadores de acesso em Java são:

- *private*: O nível máximo de encapsulamento. O membro só é visível dentro da própria classe. É aqui onde os atributos ficam protegidos.
- *protect*: Permite acesso dentro do mesmo pacote e por subclasses (herança).
- *public*: O nível mínimo de encapsulamento. Qualquer classe pode acessar o membro. Usado para os métodos que formam a “interface” pública do objeto.
- *default* (sem modificador): Acesso restrito ao pacote onde a classe reside.

Portanto, é prática comum definir atributos com o modificador de acesso *private* e métodos como *public*, embora essa não seja uma regra rígida, mas sim uma convenção de segurança. Um exemplo clássico para ilustrar a aplicação desses modificadores é a classe *ContaBancaria*. O atributo *saldo*, por exemplo, não deve ser modificado diretamente por outras classes, pois isso permitiria a manipulação arbitrária de valores, ferindo a integridade dos dados. Para impedir essa situação, declaramos o atributo *saldo* como *private*. Dessa forma, qualquer alteração externa deve ocorrer obrigatoriamente por meio de métodos *public*, os quais podem implementar as validações necessárias para assegurar as regras de negócio do sistema. Observe o exemplo abaixo:

```
public class ContaBancaria {  
    // Encapsulamos o atributo para proteger a integridade  
    private double saldo;  
    // Fornecemos acesso controlado via métodos (Interface Pública)  
    public void depositar(double valor) {  
        if (valor > 0) {  
            this.saldo += valor;  
        }  
    }  
    public double getSaldo() {  
        return this.saldo;  
    }  
}
```

```
}
}
```

Voltando ao nosso exemplo da clínica veterinária, vamos ajustar os modificadores de acesso dos atributos e métodos para aplicar o encapsulamento. Adicionaremos à classe `Pet` os métodos `registrarPet()`, `listarPet()` e `buscarPet()`, que compõem operações fundamentais do padrão CRUD (*Create* - Criar, *Read* - Ler, *Update* - Atualizar e *Delete* - Deletar). Por enquanto, estes métodos exibirão apenas mensagens de console, simulando a execução das ações que serão implementadas detalhadamente nos próximos capítulos.

```
public class Pet {
    // Atributos com encapsulamento (private)
    private String nomeAnimal;
    private int idadeAnimal;
    private int sexoAnimal;
    private boolean vacinado;

    public void vacinar() {
        this.vacinado = true;
        System.out.println(this.nome + " foi vacinado com sucesso!
        ");
    }
    // Métodos (CRUD) com implementação simplificada (stubs)
    public void registrarPet() {
        System.out.println("Ação: Registrando novo pet no sistema
        ...");
    }

    public void listarPet() {
        System.out.println("Ação: Listando pets cadastrados...");
    }

    public void buscarPet() {
        System.out.println("Ação: Buscando pet no banco de dados
        ...");
    }
}
```

Dessa forma outras classes só terão acesso aos métodos públicos, enquanto os atributos permanecem protegidos, garantindo a integridade e consistência dos dados. Na próxima seção vamos ver como acessar os atributos de forma controlada, utilizando os métodos getters e setters.

4.4.2 Getters e Setters

Com a aplicação de modificadores de acesso restritivos (como o `private`), não é possível acessar diretamente os atributos de um objeto a partir de outras classes. Na prática, contudo, precisamos manipular esses dados de forma controlada; para isso, utilizamos métodos específicos. Por convenção, os métodos destinados a retornar o valor de um

atributo iniciam com o prefixo *get*, enquanto aqueles destinados a alterar seu conteúdo utilizam o prefixo *set* em sua nomenclatura. Veja o exemplo para nossa classe Pet.

```
public class Pet {
    // Atributos com encapsulamento (private)
    private String nomeAnimal;
    private int idadeAnimal;
    private int sexoAnimal;
    private boolean vacinado;

    public String getNomeAnimal() {
        return nomeAnimal;
    }
    public void setNomeAnimal(String nomeAnimal) {
        this.nomeAnimal = nomeAnimal;
    }
    public int getIdadeAnimal() {
        return idadeAnimal;
    }
    public void setIdadeAnimal(int idadeAnimal) {
        this.idadeAnimal = idadeAnimal;
    }
    public int getSexoAnimal() {
        return sexoAnimal;
    }
    public void setSexoAnimal(int sexoAnimal) {
        this.sexoAnimal = sexoAnimal;
    }
    public boolean isVacinado() {
        return vacinado;
    }
    public void vacinar() {
        this.vacinado = true;
        System.out.println(this.nome + " foi vacinado com sucesso!
        ");
    }
    // métodos (CRUD) com implementação simplificada (stubs)
    public void registrarPet() {
        System.out.println("Ação: Registrando novo pet no sistema
        ...");
    }
    public void listarPet() {
        System.out.println("Ação: Listando pets cadastrados...");
    }

    public void buscarPet() {
        System.out.println("Ação: Buscando pet no banco de dados
        ...");
    }
}
```

Observação

É uma má prática criar uma classe e, logo em seguida, criar getters e setters para todos seus atributos. Você só deve criar um getter ou setter se tiver a real necessidade.

Outras vantagens do uso de getters e setters incluem:

- Validação de Atributos: No método setter, você pode adicionar regras de validação antes de atribuir o valor. Por exemplo, garantir que a idade não seja um número negativo ou que um saldo bancário não receba valores inválidos.
- Controle de Acesso (Leitura ou Escrita): Você pode criar propriedades somente leitura (criando apenas o getter e omitindo o setter) ou somente escrita, controlando exatamente como os dados são expostos.
- Flexibilidade e Manutenibilidade: Se você precisar mudar a forma como um dado é armazenado ou calculado internamente, basta alterar o código dentro do getter/setter. O restante do sistema que utiliza esses métodos não precisará ser reescrito, pois a interface externa continua a mesma.

Voltando ao nosso exemplo, agora vamos modificar nossa classe principal, para ver os métodos públicos getters e setters.

```
public class Principal {
    public static void main(String[] args) {
        // 1. Instanciação do objeto
        Pet meuPet = new Pet();

        // 2. Uso dos métodos SET para definir os atributos
        meuPet.setNomeAnimal("Rex");
        meuPet.setIdadeAnimal(3);
        meuPet.setSexoAnimal(1); // Ex: 1 para macho, 2 para fêmea

        // 3. Uso dos métodos CRUD
        meuPet.registrarPet();

        // 4. Ação específica: vacinar
        System.out.println("Status atual antes da vacina: " +
            meuPet.isVacinado());
        meuPet.vacinar();

        // 5. Uso dos métodos GET para exibir informações
        System.out.println("Nome do Pet: " + meuPet.getNomeAnimal());
        System.out.println("Idade do Pet: " + meuPet.getIdadeAnimal());
    }
}
```

```

        System.out.println("Status apos vacina: " + meuPet.
            isVacinado());

        // 6. Outros métodos de simulação
        meuPet.listarPet();
        meuPet.buscarPet();
    }
}

```

4.4.3 Construtores

Quando usamos o comando **new**, estamos construindo um objeto e ele chama o construtor da classe. O construtor da classe é semelhante a um método, apesar da semelhança ele não é considerado um método propriamente dito. Para criar um construtor utilizamos **mesmo nome da classe** (observe o exemplo abaixo)(Cabral (2014)).

```

...
    public Pet(){ }
...

```

Em nenhum exemplo dessa apostila até então foi criado um construtor. Então como o objeto foi criado na classe principal? Se não existir esse método, Java cria ele automaticamente, chamamos isso de *construtor default*. Um construtor default não possui argumentos (parâmetros), ou seja o objeto inicia com os atributos nulos (NULL).

O uso de um construtor se justifica em várias situações, como, por exemplo, desejar-se que os objetos sejam iniciado pelo usuário com valores que são obrigatórios. Veja o exemplo abaixo.

```

...
    public Pet(String nome){
        this.nomeAnimal = nome;
    }

```

Nesse exemplo, obrigamos a criação do objeto Pet com nome do animal. A ainda uma situação onde um objeto só faz sentido existir se outro objeto existir (será discutido isso no próximo capítulo), nesse caso precisamos passar como argumento esse objeto.

4.4.4 Sobrecarga e assinatura de métodos

Em Java, a definição de um comportamento não se resume apenas ao nome que damos ao método. Para o compilador, a identidade de uma ação é definida pela sua assinatura.

Métodos não são definidos apenas pelo nome: O nome é apenas o identificador primário. Para diferenciar funcionalidades, o Java utiliza um conjunto de características chamado assinatura.

Assinatura de um método define o seu “nome completo”: É a assinatura que permite ao compilador saber exatamente qual bloco de código deve ser executado quando um método é chamado.

Fazem parte da assinatura de um método:

- Nome do método: A identificação básica.
- Quantidade de parâmetros: Quantos dados o método recebe.
- Tipos dos parâmetros: Quais são os tipos de dados (ex: int, String, float).
- Ordem dos parâmetros: A sequência em que os tipos são declarados.

São métodos diferentes, apesar do mesmo nome: Quando mantemos o nome, mas alteramos os parâmetros, criamos variações da mesma ação:

```
public void metodo()  
  
public void metodo(int a)  
  
public void metodo(float a)  
  
public void metodo(int a, float b)  
  
public void metodo(float a, int b)
```

Essa característica da programação orientada a objetos damos o nome de sobrecarga ou *Overloading*. Ela oferece flexibilidade e legibilidade, permitindo que o desenvolvedor utilize o mesmo nome para operações que, embora semanticamente similares, aceitam entradas de dados diferentes.

Logo, podemos ter vários construtores, com diferentes assinaturas utilizando essa propriedade.

Observação

O **overloading** é uma característica da programação orientada a objetos que permite a criação de múltiplos métodos com o mesmo nome, mas com diferentes assinaturas.

4.5 Métodos e atributos estáticos

Nessa seção será apresentado o conceito de métodos comumente utilizados e atributos estáticos.

4.5.1 Sobre escrita *Override*

Antes de abordar outros métodos vamos falar sobre a sobre escrita de métodos, ou *override*, este é um conceito na programação orientada a objetos que permite que sobrescrever um método de outra classe. Em Java a sobrescrita é feita utilizando a anotação `@Override` antes da declaração do método. Essa anotação não é obrigatória, mas é

altamente recomendada, pois ajuda o compilador a verificar se você está realmente sobrescrevendo um método existente na superclasse. Se houver algum erro, como uma assinatura incorreta, o compilador emitirá um aviso.

4.5.2 Método toString

O método `toString()` é um componente fundamental da classe `Object` em Java. Por padrão, quando tentamos exibir um objeto diretamente, o Java retorna um código hexadecimal pouco compreensível (como `Pet@15db9742`). Portanto, o método `toString()` é extremamente útil para exibir as informações de um objeto de forma legível para humanos. Para que essa representação seja útil no nosso sistema de pets, podemos sobrescrever esse método em nossas classes personalizadas, garantindo que ele retorne os dados que realmente importam.

```
public class Pet {
    private String nomeAnimal;
    private int idadeAnimal;
    private int sexoAnimal;

    public Pet(String nome, int idade, int sexo) {
        this.nomeAnimal = nome;
        this.idadeAnimal = idade;
        this.sexoAnimal = sexo;
    }
    // Sobrescrita do método, abordaremos no próximo capítulo
    @Override
    public String toString() {
        return "Pet [Nome: " + nomeAnimal + ", Idade: " +
            idadeAnimal + " anos, " + Sexo: " + sexoAnimal]";
    }
}

// No uso do sistema:
Pet meuPet = new Pet("Rex", 3);
System.out.println(meuPet);
// Saída no console: Pet [Nome: Rex, Idade: 3 anos, Sexo: 1]
```

4.5.3 Método equals

Como descrito no início do capítulo, a identidade do objeto é única e imutável, portanto, é muito difícil de comparar dois objetos, mesmo que eles possuam os mesmos atributos e valores. Para isso, o Java fornece o método `equals()` que compara se dois objetos são iguais, ou seja, se possuem os mesmos valores de atributos.

Para utilizar esse método, precisamos, sobrescrever o método `equals()` na classe que desejamos comparar. Abaixo segue um exemplo de como implementar o método `equals()` na classe `Pet`.

```

public class Pet {
    private String nomeAnimal;
    private int idadeAnimal;
    private int sexoAnimal;
    private boolean vacinado;

    // Construtor e outros métodos omitidos

    @Override
    public boolean equals(Object obj) {
        if (obj instanceof Pet) {
            Pet p = (Pet) obj;
            return this.nomeAnimal.equals(p.nomeAnimal) &&
                this.idadeAnimal == p.idadeAnimal &&
                this.sexoAnimal == p.sexoAnimal;
        }
        return false;
    }
}

```

Observação

O operador `instanceof` em Java é uma palavra-chave utilizada para realizar testes de tipo em tempo de execução (*runtime type checking*). Ele permite verificar se um objeto é uma instância de uma classe específica, se implementa uma determinada interface ou se pertence a uma subclasse.

4.5.4 Atributos estáticos

Até agora nesse capítulo utilizamos métodos e atributos de instância, ou seja, cada objeto possui seus próprios valores. No entanto, existem situações em que queremos que um atributo ou método seja compartilhado entre todos os objetos de uma classe. Para isso, utilizamos o modificador `static`. Atributos e métodos estáticos pertencem à classe em si, e não a instâncias individuais. Isso significa que eles podem ser acessados diretamente através do nome da classe, sem a necessidade de criar um objeto.

No exemplo da clínica, não faz muito sentido o uso de atributos estáticos, mas um exemplo seria manter o número de pets cadastrados.

```

...
    private static int contadorPets = 0;
...

```

Observação

Por isso, eles só podem acessar outros membros estáticos da mesma classe. Essa restrição existe porque, ao invocar um método estático, não há uma instância

associada — ou seja, não temos acesso à referência *this*. Como o *this* representa o objeto em execução e métodos estáticos são chamados diretamente via nome da classe, torna-se impossível acessar dados que dependeriam de um objeto particular para existir (Cabral (2014)).

Um novo exemplo de uso de atributos estáticos é a classe `Math` do Java, que contém métodos e constantes matemáticas. Por exemplo, a constante `Math.PI` representa o valor de pi e é acessada diretamente pela classe, sem a necessidade de criar uma instância da classe `Math`. Veja o exemplo abaixo:

```
public class ExemploMath {
    public static void main(String[] args) {
        double raio = 5.0;
        double area = Math.PI * Math.pow(raio, 2); // nao e
            necessario criar um objeto da classe Math para acessar
            a constante PI e o metodo pow()
        System.out.println("Area do circulo: " + area);
    }
}
```

4.6 Representação de classes na UML (Linguagem de Modelagem Unificada)

A UML é uma linguagem visual padronizada para documentar, especificar e projetar sistemas. Essencial em engenharia de software, ela transforma lógicas e códigos complexos em diagramas de fácil compreensão, divididos entre visões estruturais e comportamentais. Dentro da UML existe o diagrama de classes.

Uma classe é representada por um retângulo dividido em Nome, Atributos e Métodos. Mais tarde serão adicionados sinais que definem o tipo de encapsulamento dos atributos e métodos. A Figura 4.5 representa a classe `Pet`.

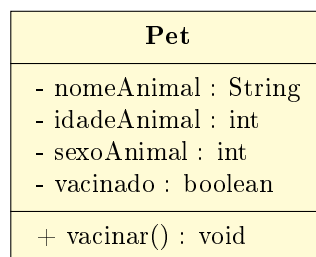


Figura 4.5: Diagrama UML da classe `Pet`.

A UML não é foco desta apostila, mas é interessante representar as classes de forma padronizada. Ao longo dos próximos capítulos e seções serão inseridos novos tipos de representações.

4.7 Resumo dos principais tópicos abordados

A tabela 4.1 abaixo resume os principais conceitos abordados neste capítulo, fornecendo uma visão geral das definições e exemplos discutidos.

Tabela 4.1: Resumo dos conceitos fundamentais de POO

Termo	Breve Explicação
Abstração	Generalização das características essenciais de um objeto para resolver um problema.
Objeto	Representação concreta de uma entidade; possui estado (atributos) e comportamento (métodos).
Classe	O "molde" ou modelo que define os atributos e comportamentos dos objetos.
Instância	O objeto concretizado em memória, criado a partir de uma classe.
Encapsulamento	Técnica de ocultar detalhes internos, expondo apenas uma interface pública e segura.
Atributos	As características ou dados que definem o estado de um objeto.
Métodos	As ações ou comportamentos que um objeto pode realizar.
Construtor	Método especial chamado ao instanciar um objeto para inicializar seu estado.
Sobrecarga	Capacidade de ter múltiplos métodos com o mesmo nome, mas assinaturas (parâmetros) diferentes.
Static	Modificador que faz atributos/métodos pertencerem à classe, não a uma instância específica.

5 Relacionamentos entre classes

Uma das maiores vantagens da Orientação a Objetos é o reuso de código, que permite aproveitar implementações existentes para criar novas funcionalidades. Além disso, a POO facilita a modelagem de sistemas complexos através do relacionamento entre objetos, permitindo que eles colaborem entre si para executar tarefas maiores.

5.1 Herança

Considere a necessidade de cadastrar os proprietários dos pets em nosso sistema, utilizando os atributos **nome**, **cpf** e **e-mail**. Paralelamente, precisamos gerenciar os profissionais da clínica, como os veterinários, que compartilham os mesmos dados básicos de identificação, mas possuem atributos adicionais, como o **CRMV** (Conselho Regional de Medicina Veterinária).

Fica o questionamento, precisamos criar duas classes praticamente idênticas só pela diferença do atributo CRMV? E se existir a necessidade de adicionar um novo atributo comum à cliente e veterinário, teríamos que percorrer todas classes para realizar essa mudança?

Com a herança, é possível economizar linhas de código e consequentemente tempo desenvolvendo programas baseando novas classes existentes em classes existente. Ao criar uma classe, em vez de declarar membros novos, você pode designar que a nova classe, chamada de **subclasse**, deva **herdar** membros de uma classe existente, que é chamada de **superclasse** (Deitel and Deitel (2015)).

Observação

Podemos dizer que um objeto **é um tipo de** outro objeto, quando precisamos utilizar herança.

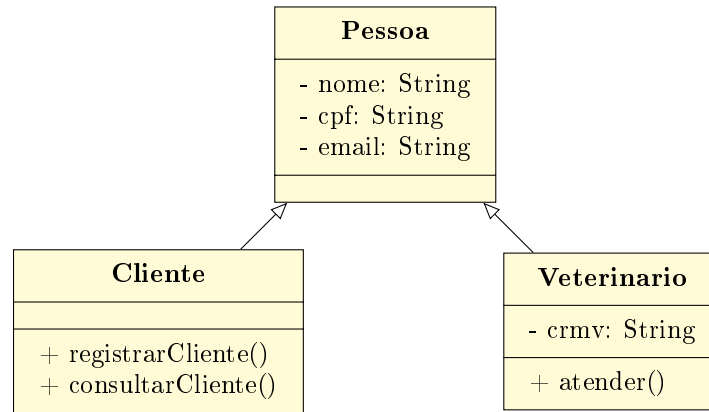
5.1.1 Tipos de herança

Na POO Herança possui dois tipos:

- **Simples:** Ocorre quando uma subclasse possui apenas uma superclasse. O objetivo é especializar e reutilizar membros de um único conceito de classe base. Veja o exemplo abaixo
- **Herança Múltipla:** Ocorre quando uma subclasse necessita de duas ou mais superclasses, permitindo especializar mais de um conceito da aplicação simultaneamente. Novos exemplos serão apresentados nos próximos capítulos.

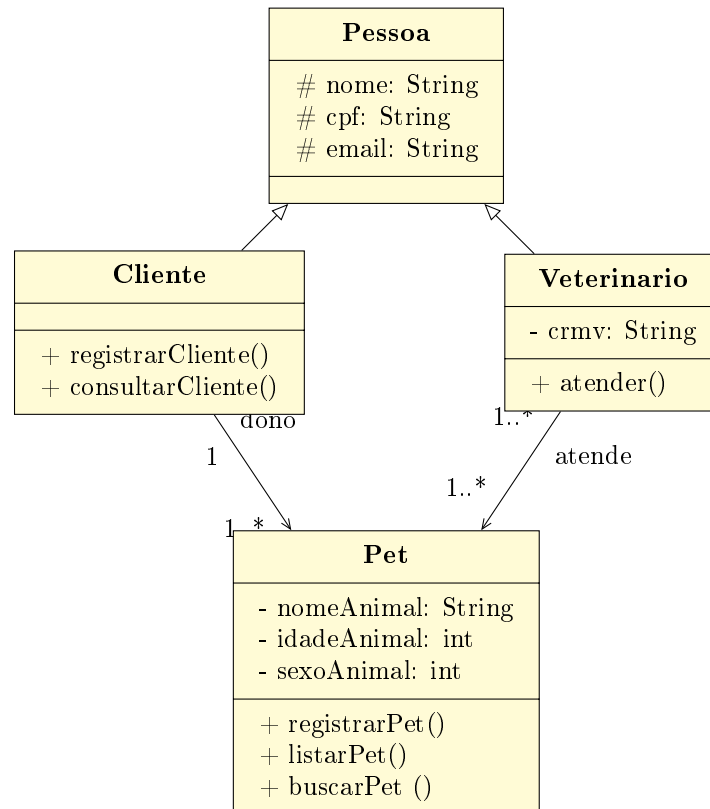
5.1.2 Herança simples

Antes de iniciarmos a implementação, vamos entender como a herança funciona na UML. Uma herança, na UML, é representada por seta entre uma subclasse e uma superclasse. Para representar os modificadores de acesso (4.4.1) usamos – para private, + para public e # para protect.



Em herança simples, uma subclasse tem a capacidade de estender a superclasse, incorporando seus próprios atributos e métodos exclusivos. Por essa razão, ela é considerada mais específica e representa uma **especialização** da classe original. Além de herdar os comportamentos da superclasse, a subclasse pode **sobrescrevê-los** para adaptá-los às suas necessidades particulares, reforçando o conceito de que a herança funciona como um mecanismo de **especialização** (Deitel and Deitel (2015)). No exemplo UML abaixo vamos integrar nossa classe Pet ao diagrama. Retiramos o atributo vacinado, pois utilizamos apenas de forma didática anteriormente.

5 Relacionamentos entre classes



Para acessar os atributos herdados sem comprometer o encapsulamento utilizamos o modificador de acesso **protected**, que se posiciona entre o **private** e o **public** em termos de visibilidade. Um atributo ou método **protected** é acessível pela própria classe e por suas subclasses, garantindo que o acesso seja restrito à hierarquia de herança, mantendo a integridade dos dados protegida contra acessos externos não autorizados.

Observação

Em Java, para uma classe herdar outra classe utilizamos a palavra **extends**, após o nome da classe seguido do nome da superclasse.

Veja o exemplo abaixo, sobre a herança simples em Java.

```
// Superclasse
class Pessoa {
    // Usamos protected para permitir acesso pelas subclasses,
    // mas esconder do restante do sistema.
    protected String nome;
    protected String cpf;
    protected String email;

    public Pessoa(String nome, String cpf, String email) {
        this.nome = nome;
    }
}
```

```

        this.cpf = cpf;
        this.email = email;
    }
}

// Subclasse
class Veterinario extends Pessoa {
    private String crmv;

    public Veterinario(String nome, String cpf, String crmv) {
        super(nome, cpf, email); // Chama o construtor da
        superclasse
        this.crmv = crmv;
    }

    public void exibirInformacoes() {
        // Acesso direto aos atributos protected herdados
        System.out.println("Nome: " + this.nome);
        System.out.println("CPF: " + this.cpf);
        System.out.println("CRMV: " + this.crmv);
    }
}

```

Além do exemplo apresentar o uso do **extends**, a palavra **super** é usada para chamar o construtor da superclasse. Ainda no exemplo, pode-se observar o acesso aos atributos da classe “mãe”, pois seu acesso é do tipo *protected*. Para o acesso, basta usar a palavra **this**.

Observação

Por uma questão de organização, é boa prática criar um arquivo .java para cada classe.

5.1.3 Herança múltiplas

A herança múltipla ocorre quando uma subclasse necessita não de apenas uma, mas duas ou mais classes para herdar. Leite et al. (2016). Dessa forma a classe filha herda os atributos e métodos de todas as classes que a compõem. A herança múltipla é um recurso muito avançado e deve ser usado com cautela, pois pode introduzir complexidade e ambiguidade no design do sistema.

O exemplo abaixo de uma representação de herança múltipla na UML, onde a classe **CaoGuarda** herda de duas superclasses, **Pet** e **AnimalTrabalho**. A classe **CaoGuarda** possui atributos e métodos próprios, além de herdar os atributos e métodos das superclasses. A seta indica a relação de herança entre as classes, mostrando que **CaoGuarda** é uma especialização de ambas as superclasses.

5 Relacionamentos entre classes

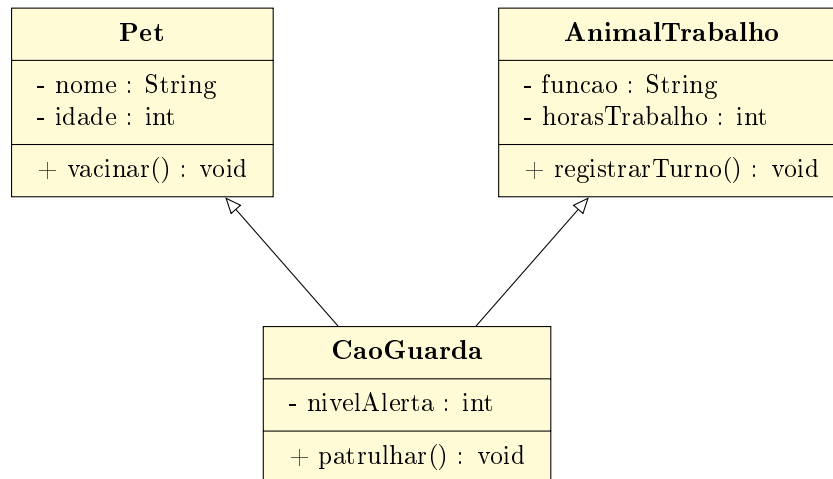


Figura 5.1: Exemplo conceitual de herança múltipla em UML entre classes.

Observação

Em Java, a herança múltipla de classes não é suportada diretamente, mas pode ser alcançada através do uso de interfaces, assunto esse que veremos no próximo capítulo.

Exercícios

1. O objetivo deste exercício é compreender como a **herança**. Utilizaremos o exemplo de uma hierarquia acadêmica, onde **Aluno** e **Professor** herdam características de **Pessoa**.

Superclasse (Pessoa)

Desenvolva a classe **Pessoa** como a base da hierarquia, contendo os seguintes atributos privados:

Identificador	Tipo	Descrição
nome	String	Nome da pessoa
idade	int	Idade da pessoa

A classe deve conter métodos **get** para os atributos privados e o seguinte construtor público:

Identificador	Retorno	Parâmetro	Descrição
construtor	–	nome, idade	Inicializa os atributos da superclasse

Subclasses (Aluno e Professor)

Implemente as subclasses utilizando a palavra-chave **extends**:

- **Classe Aluno:** Adiciona o atributo privado **curso** (**String**). Utilize **super(nome, idade)** no construtor para inicializar os atributos herdados.

Identificador	Tipo	Descrição
curso	String	Curso em que o aluno está matriculado

Identificador	Retorno	Parâmetro	Descrição
construtor	–	nome, idade, curso	Inicializa atributos da superclasse e o curso
toString	String	–	Retorna representação textual formatada do aluno

- **Classe Professor:** Adiciona o atributo privado **matricula** (**int**). Utilize **super** para delegar a inicialização dos dados herdados.

Identificador	Tipo	Descrição
matricula	int	Número de matrícula do professor

Identificador	Retorno	Parâmetro	Descrição
construtor	–	nome, idade, matrícula	Inicializa atributos da superclasse e a matrícula
toString	String	–	Retorna representação textual formatada do professor

Classe Principal

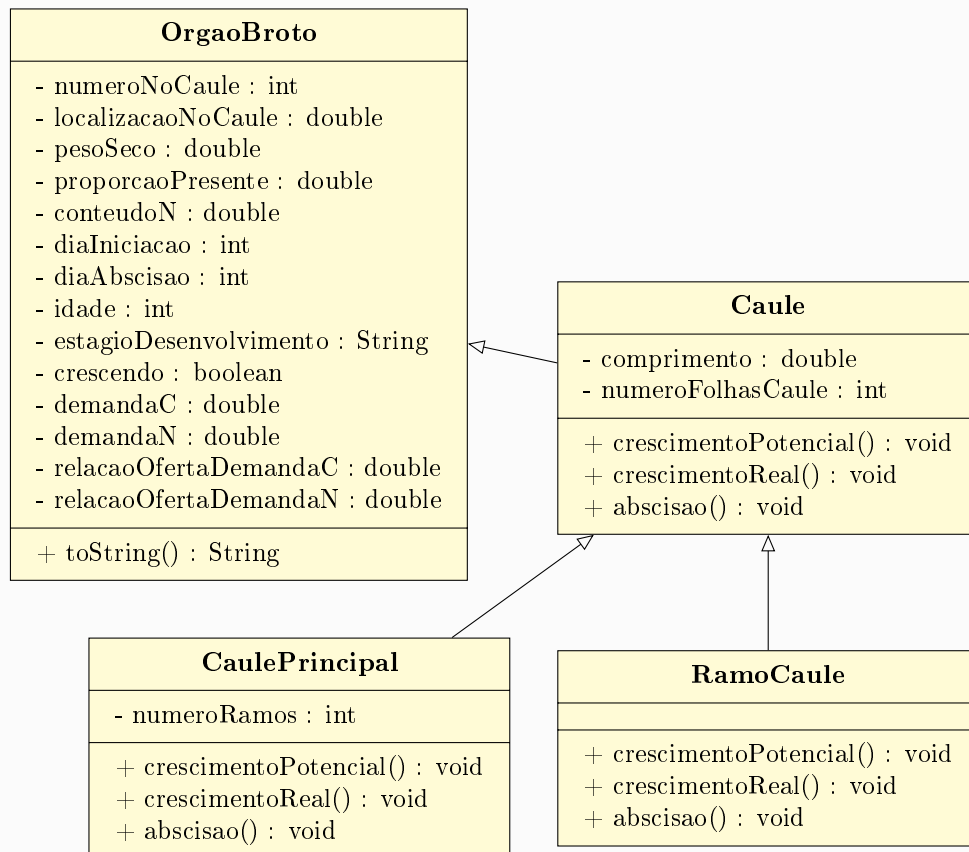
Desenvolva a classe `Classe_aluno_heranca` com o método `main` para instanciar os objetos:

- Crie um objeto `Aluno` e um objeto `Professor`.
- Exiba os dados de cada um utilizando os métodos herdados da classe `Pessoa` (como `getNome()` e `getIdade()`).
- Observe como os dados do aluno incluem o `curso`, enquanto os do professor seguem sua estrutura própria.

Reflexão

Note que, embora `Aluno` e `Professor` sejam classes distintas, ambos compartilham a mesma lógica de armazenamento de nome e idade definida em `Pessoa`. Como você poderia impedir que alguém criasse um objeto do tipo `Pessoa` diretamente, tornando-a uma classe puramente de modelo?

- Considere o diagrama UML de uma hierarquia voltada para a modelagem de órgãos vegetais em um sistema agrícola. Desenvolva as classes com os atributos e métodos traduzidos conforme as especificações a seguir.



Superclasse (OrgaoBroto / *ShootOrgan*)

Desenvolva a classe base contendo os seguintes atributos privados (representando dados de desenvolvimento e demanda):

Identificador	Tipo	Descrição
numeroNoCaule	int	Número do caule associado
localizacaoNoCaule	double	Localização no caule
pesoSeco	double	Peso seco do órgão
proporcaoPresente	double	Proporção presente do órgão
conteudoN	double	Conteúdo de nitrogênio (N)
diaIniciacao	int	Dia de iniciação do órgão
diaAbscisao	int	Dia de abscisão (queda)
idade	int	Idade do órgão em dias
estagioDesenvolvimento	String	Estágio atual de desenvolvimento
crescendo	boolean	Indicador se está em crescimento
demandaC	double	Demanda de carbono (C)
demandaN	double	Demanda de nitrogênio (N)
relacaoOfertaDemandaC	double	Relação de oferta e demanda de carbono
relacaoOfertaDemandaN	double	Relação de oferta e demanda de nitrogênio

O construtor da classe deve inicializar os seguintes atributos:

Identificador	Retorno	Parâmetro	Descrição
construtor	–	numeroNoCaule, diaIniciacao, estagioDesenvolvimento	Inicializa atributos da classe

A classe deve conter métodos **get** e **set** para os atributos e o seguinte método público:

Identificador	Retorno	Parâmetro	Descrição
toString	String	–	Retorna a representação textual dos dados do órgão

Superclasse intermediária (Caule / *Stem*) e Subclasses

Implemente a classe **Caule** que herda de **OrgaoBroto**, adicionando atributos específicos de caule, e crie as especializações **CaulePrincipal** e **RamoCaule**:

- **Classe Caule:** Adiciona os atributos privados **comprimento** (double) e **numeroFolhasCaule** (int). Utilize **super** no construtor.
- **Classe CaulePrincipal (estende Caule):** Adiciona o atributo **numeroRamos** (int) e implementa os comportamentos de crescimento.
- **Classe RamoCaule (estende Caule):** Herda a estrutura de **Caule** e implementa os métodos específicos de crescimento e abscisão.

Dica

Os construtores das subclasses devem utilizar **super** para delegar a inicialização dos atributos herdados da superclasse. O construtor da classe **caule** deve inicializar os atributos **comprimento** e **numeroFolhasCaule**, enquanto os construtores das subclasses devem inicializar seus próprios atributos específicos e das superclasses.

Identificador	Retorno	Parâmetro	Descrição
crescimentoPotencial	void	–	Calcula o crescimento potencial do caule
crescimentoReal	void	–	Calcula o crescimento real obtido
abscisao	void	–	Simula o processo de queda do órgão

Dica

Nos métodos faça apenas mensagens de saída (`System.out.println()`) indicando que o método foi chamado, sem implementar a lógica real de crescimento ou abscisão.

Classe Principal

Desenvolva uma classe **Principal** para instanciar um objeto do tipo **CaulePrincipal** e um do tipo **RamoCaule**, atribuindo valores iniciais por meio de construtores com **super**, testando os métodos de crescimento e exibindo as informações detalhadas de cada um por meio do método `toString()`.

5.2 Polimorfismo

Em determinados momentos em uma hierarquia de classes, precisamos que um mesmo método de uma superclasse que foi herdado por uma subclasse se comporte de **forma diferente dependendo do objeto instanciado**. Isto surge devido à necessidade de flexibilidade que a hierarquia de classe deseja fornecer (Leite et al. (2016)).

A maior vantagem do uso do polimorfismo é que podemos utilizar objetos distintos e continuar executando o mesmo método, sendo que este método se moldará ao objeto corrente. Veja o exemplo abaixo, não necessariamente estará em nosso sistema final.

Observação

- O polimorfismo é um conceito que permite que objetos de diferentes classes sejam tratados de forma uniforme, desde que compartilhem uma classe base.
- Ele promove a reutilização de código e facilita a manutenção do sistema, pois permite que novas classes sejam adicionadas sem alterar o código existente.

```
// Superclasse
class Pessoa {
    protected String nome;

    public void exibirPerfil(); // Metodo polimorfico
}

// Subclasse 1: Cliente
class Cliente extends Pessoa {
    @Override
    public void exibirPerfil() {
        System.out.println("Perfil do Cliente: " + this.nome);
    }
}

// Subclasse 2: Veterinário
class Veterinario extends Pessoa {
    @Override
    public void exibirPerfil() {
        System.out.println("Perfil do Veterinário: " + this.nome);
    }
}
```

O ato de redefinir um método herdado na subclasse é chamado de sobrescrita ou *override*. Diferentemente da sobrecarga (*overload*), em que métodos possuem o mesmo nome dentro da mesma classe, a sobrescrita ocorre com métodos de mesma assinatura situados em classes diferentes na hierarquia de herança. Assim, cada subclasse pode fornecer uma funcionalidade específica para aquele método, adaptando-o às necessidades de cada objeto.

Observação

A sobreescrita (*override*) de métodos é a prática para implementar o polimorfismo, permitindo que métodos de classes diferentes respondam de forma distinta utilizando a mesma assinatura.

5.2.1 Final

Muitas vezes não queremos que uma classe seja estendida, um método seja sobrescrito ou uma variável seja alterada. Para isso, utilizamos a palavra **final** em Java.

A Palavra **final** em Java pode ser considerada um modificador de acesso que indica que um elemento não pode ser alterado após sua definição. Sua aplicação pode estar em diferentes contextos, como variáveis, métodos e classes.

- Variáveis final (Constantes): O valor da variável não pode ser modificado após a sua atribuição inicial. Se for um tipo primitivo, o valor é imutável; se for um objeto, a referência não pode apontar para outro objeto (embora o estado interno do objeto ainda possa ser alterado, a menos que ele seja imutável). É comum usá-la com **static** para criar constantes globais (`public static final int MAX_USUARIOS = 100;`).
- Métodos final (Impedem Sobreescrita): O método não pode ser reescrito (`@Override`) pelas subclasses. Isso é útil quando a implementação atual é definitiva e alterar sua lógica comprometeria a segurança ou o funcionamento correto do sistema.
- Classes final (Impedem Herança): A classe não pode ser estendida por nenhuma outra (não aceita a palavra `extends`). Um exemplo clássico na biblioteca padrão do Java é a classe `String`, que é final para garantir segurança e imutabilidade estrutural.

A sintaxe para declarar um elemento como final é simples, basta adicionar a palavra-chave **final** antes da declaração do elemento. Por exemplo:

```
public final class MinhaClasseFinal {
    // Atributo estático e final
    public static final int MAX_USUARIOS = 100;
    //método final
    public final void metodoFinal() {
        // Implementação do método
    }
    //método estático final
    public static final void metodoEstaticoFinal() {
        // Implementação do método estático final
    }
}
```

Observação

Pode-se combinar o comando final com métodos do tipo static, criando métodos que não podem ser sobrescritos e que pertencem à classe em vez de instâncias específicas. Isso é útil para utilitários ou funções auxiliares que não dependem do estado de um objeto.

5.3 Associação

Existem situações onde a herança não é a melhor escolha de relacionamento, lembrando que quando utilizamos herança dizemos que um objeto filho (subclasse) é um tipo do objeto específico da classe mãe (superclasse). E se precisarmos cadastrar o endereço dos veterinários e clientes? Conceitualmente, o endereço não é um tipo de cliente e ou veterinário, muito menos um tipo de pessoa, logo o relacionamento entre eles é apenas de “ajuda”. Este tipo de relacionamento é chamado de associação.

Existem três tipos de associações que variam de acordo com dependência de um objeto com outro.

5.3.1 Associação (Geral)

É o relacionamento mais amplo, onde uma classe utiliza ou conhece a outra para realizar suas funções. Não implica, necessariamente, dependência de ciclo de vida.

Como exemplo, o veterinário atende um Pet. Ambos existem de forma independente, mas o veterinário possui uma associação temporária com o animal durante a consulta.

Um exemplo de código em Java para representar a associação entre um Veterinário e um Pet é apresentado abaixo:

```
class Veterinario {
    private String nome;
    //Demais atributos da classe Veterinário ...
    // Associação: Veterinário ‘tem um’ Pet
    private Pet pet;

    public Veterinario(String nome) {
        this.nome = nome;
    }

    public void atenderPet(Pet pet) {
        this.pet = pet;
        System.out.println(“Veterinário ” + this.nome + “ está
            atendendo o pet ” + pet.getNome());
    }
}
```

A associação geral é representada na UML por uma linha simples conectando as classes envolvidas, indicando que elas estão relacionadas de alguma forma, mas sem especificar a

natureza exata dessa relação. A Figura 5.2 ilustra a associação entre as classes Veterinário e Pet.

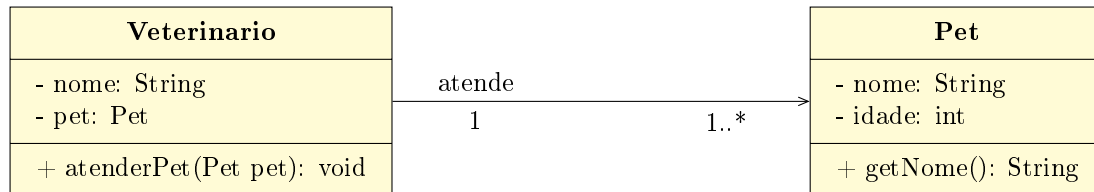


Figura 5.2: Exemplo de associação entre as classes Veterinário e Pet na UML.

5.3.2 Agregação

Representa um relacionamento “todo-parte”, mas é uma forma fraca de associação. As classes podem existir independentemente uma da outra. Se o “todo” for destruído, as “partes” continuam existindo.

Exemplo: Um Cliente e seus Pets. Se o cliente for removido do sistema, o objeto Pet pode persistir (por exemplo, vinculado a outro responsável ou ao histórico da clínica).

Vaja o código abaixo, representando a agregação entre Cliente e Pet:

```

public class Pet {
    private String nome;
    private String especie;
    //Outros atributos da classe Pet...
    public Pet(String nome,
               String especie) {
        this.nome = nome;
        this.especie = especie;
    }

    public String getNome() {
        return this.nome;
    }
}
  
```

```

public class Cliente {
    private String nomeCliente;
    // 0 pet existe independentemente
    private Pet pet;
    //construtor da classe Cliente solicitando um objeto Pet como
    //parâmetro
    public Cliente(String nome,
                   Pet pet) {
        this.nomeCliente = nome;
        this.pet = pet;
    }
}
  
```

```

public void exibirDados() {
    System.out.println('Cliente: ' +
        this.nomeCliente +
        ' | Pet: ' + pet.getNome());
}
}

```

Na UML, a agregação é representada por uma linha conectando as classes, com um losango vazio na extremidade da classe “todo”, indicando que a classe “parte” pode existir independentemente da classe “todo”. A Figura 5.3 ilustra a agregação entre as classes Cliente e Pet.

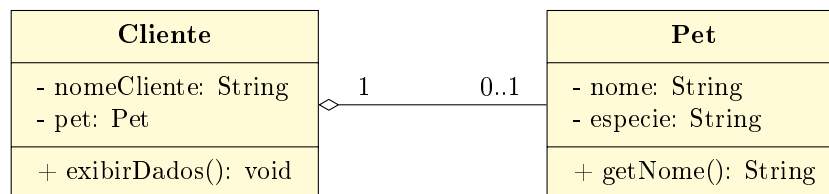


Figura 5.3: Representação UML da agregação entre as classes Cliente e Pet.

5.3.3 Composição

A composição é uma forma forte de associação, onde o “todo” é responsável pelo ciclo de vida das “partes”. Se o “todo” for destruído, as “partes” também são destruídas automaticamente. Nesse tipo de relacionamento, as “partes” não podem existir independentemente do “todo”.

Exemplo: O exemplo abaixo mostra a composição entre a classe Pessoa e a classe Endereco. O endereço é criado dentro da classe Pessoa, e se a instância de Pessoa for destruída, o endereço associado também será destruído. Além disso o exemplo mostra todos os tipos de relacionamento entre classes, como associação, agregação e composição.

```

import java.util.ArrayList;
import java.util.List;

// --- Composicao ---
class Endereco {
    private String rua;
    private String cidade;
    // O endereco nao existe fora da Pessoa (dependencia total)
    public Endereco(String rua, String cidade) {
        this.rua = rua;
        this.cidade = cidade;
    }
}

```

```

class Pessoa {
    protected String nome;
    protected String cpf;
}

```

5 Relacionamentos entre classes

```
protected String email;
// Composicao: Pessoa ‘tem um’ Endereco
private Endereco endereco;

public Pessoa(String nome, String cpf, String email, String
    rua, String cidade) {
    this.nome = nome;
    this.cpf=cpf;
    this.email=email;
    this.endereco = new Endereco(rua, cidade);
}
```

```
class Consulta {
    private String data;
    // Agregacao: Consulta ‘tem varios’ Pets (o Pet existe
    // independentemente)
    private List<Pet> petsAtendidos = new ArrayList<>();

    public void adicionarPet(Pet p) {
        this.petsAtendidos.add(p);
    }
}
```

```
class Veterinario extends Pessoa {
    private String crmv;
    // Associacao de uso
    public void realizarConsulta(Consulta c) {
        System.out.println(‘‘Veterinario realizando consulta.’’);
    }
}
```

```
class Pet {
    private String nomeAnimal;
    private int idadeAnimal;
    private int sexo;
    //Demais atributos e métodos da classe Pet...
}
```

E no arquivo principal:

```
public class Main {
    public static void main(String[] args) {
        // Criando o veterinario com seu endereco (composicao)
        Veterinario vet = new Veterinario(‘‘Dr. Marcos’’, ‘‘Rua
            das Flores, 123’’, ‘‘Ponta Grossa’’);

        // Criando pets (agregados a consulta)
        Pet pet1 = new Pet(‘‘Rex’’, 3, 1);
    }
}
```

```

Consulta consulta1 = new Consulta('20/07/2026',');
consulta1.adicionarPet(pet1);

vet.realizarConsulta(consulta1);
}
}

```

Na UML a composição é representada por um losango preto, como mostrado na Figura 5.4, nela também se encontram todos os tipos de relacionamento entre classes, como associação, agregação e composição.

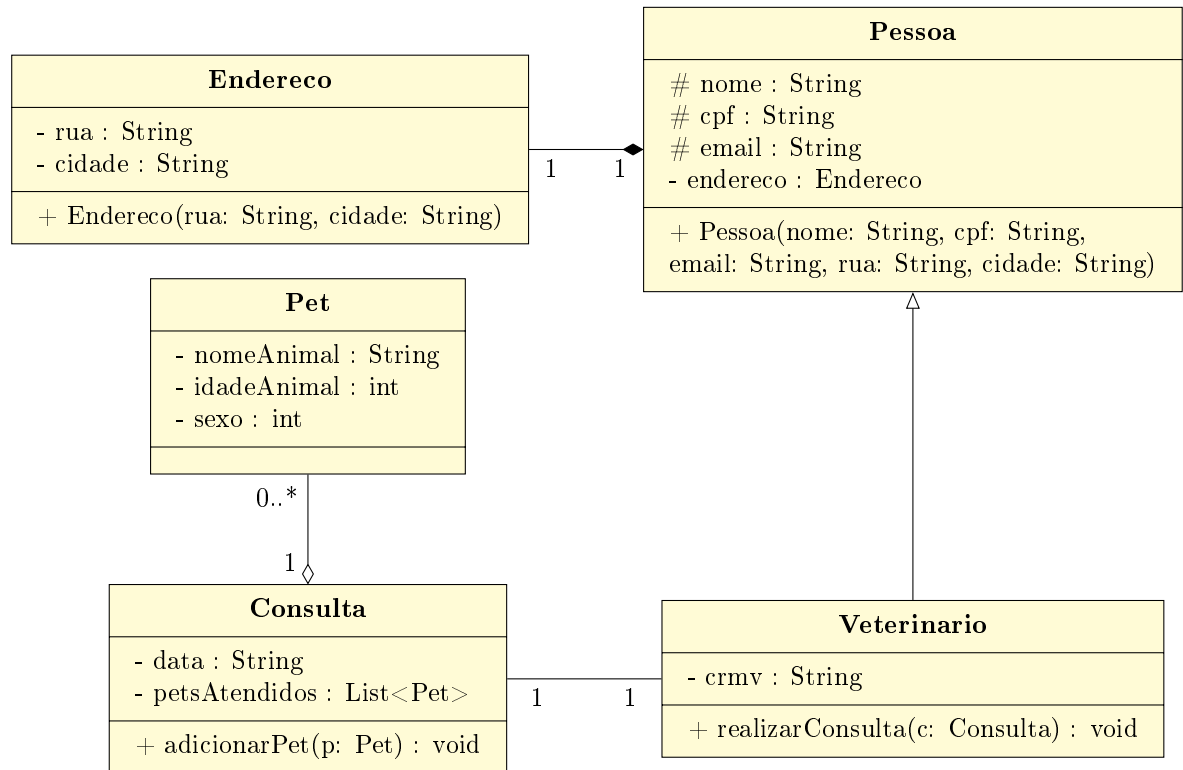


Figura 5.4: Diagrama UML representativo das classes Endereco, Pessoa, Veterinario, Consulta e Pet.

A Tabela 5.1 resume os tipos de relacionamento entre classes, suas características.

Relação	Conceito / Significado	Ciclo de Vida	Implementação em Java
Associação	Relacionamento fraco onde um objeto utiliza serviços ou dados de outro (“usa-um”).	Independentes: Os objetos existem e operam de forma totalmente autônoma.	Passagem do objeto como parâmetro de método (ex: ‘metodo(Objeto o)’).
Agregação	Relação do tipo “tem-um” (todo-parte), onde o todo contém partes, mas elas não dependem dele para existir.	Independentes: Se o objeto “todo” for destruído, a “parte” continua existindo no sistema.	Atributo de referência populado externamente, como listas ou coleções (ex: ‘List<Pet>’).
Composição	Relação de propriedade exclusiva e forte (todo-parte restrita). A parte pertence a um único todo.	Dependentes: Se o objeto “todo” for destruído, a “parte” é destruída junto obrigatoriamente.	Instanciação interna do objeto dentro do construtor do “todo” (ex: ‘this.parte = new Parte()’).

Tabela 5.1: Comparação entre Associação, Agregação e Composição.

Exercícios

Exercício: O Relacionamento de Composição

Montando um carro

1. O objetivo deste exercício é compreender o conceito de **composição** em Orientação a Objetos, modelando uma relação onde o objeto dependente é criado internamente pelo objeto principal.

Classe Motor

Desenvolva a classe **Motor** contendo o atributo privado de controle de velocidade:

Identificador	Tipo	Descrição
velocidade	int	Velocidade atual do motor

A classe deve conter os construtores e o método de incremento:

Identificador	Retorno	Parâmetro	Descrição
construtor	–	–	Padrão: inicializa velocidade com 0
construtor	–	velocidade	Inicializa velocidade com o valor passado
aumentarVelocidade	void	–	Incrementa a velocidade em 10

Classe Carro (Composição)

Desenvolva a classe **Carro** utilizando composição para incluir um objeto **Motor**:

Identificador	Tipo	Descrição
motor	Motor	Objeto motor que compõe o carro

A classe deve conter o construtor (garantindo a criação interna por composição) e o método de comportamento:

Identificador	Retorno	Parâmetro	Descrição
construtor	–	–	Instancia um novo objeto Motor internamente (composição)
acelerar	void	–	Chama o método aumentarVelocidade() do motor associado

Classe Principal

Desenvolva a classe **ExemploCarro** contendo o método **main** para testar a implementação:

- Instancie um objeto da classe **Carro**.
- Chame o método **acelerar()** do carro.
- Observe como a lógica de aceleração é delegada da classe **Carro** para a classe **Motor**.

Dica

Lembre-se: na composição, se o objeto **Carro** for destruído, o objeto **Motor** criado dentro dele também deve deixar de existir. Essa é a essência do relacionamento de composição que estamos modelando aqui.

2. Sistema de controle de validade de produto

O objetivo desta aula prática é aplicar os conceitos de Composição e Sobre-carga de Métodos (Overload) na modelagem de sistemas orientados a objetos. Ao final desta atividade, você deverá ser capaz de implementar classes que colaboram entre si através de relacionamentos estruturais, gerenciar o ciclo de vida de objetos componentes (composição) e desenvolver interfaces de classes versáteis utilizando técnicas de sobrecarga de construtores e métodos.

Classe Data

Desenvolva uma classe chamada **Data**, que contenha os seguintes atributos privados:

Atributo	Tipo	Descrição
dia	int	Dia do mês (1..31)
mes	int	Mês do ano (1..12)
ano	int	Ano (2021..)
Método	Parâmetro	Descrição
Construtor	dia, mês, ano	Inicia os valores dos três atributos.
Construtor	dia, mês	Inicia dia e mês; considera o ano de 2021.

A classe deve conter todos métodos **get** e **set** e método **toString()**.

Classe Produto

Desenvolva uma classe chamada **Produto**, que através de uma relação de **composição**, possua um atributo do tipo **Data**.

Atributos Privados

Atributo	Tipo	Descrição
nome	String	Nome do produto
preco	double	Preço de venda
quantidade	int	Quantidade em estoque
validade	Data	Objeto da classe Data
Método	Parâmetro	Descrição
Construtor	nome, qtd, preco, validade	Inicia os quatro atributos.
Construtor	nome, preco, validade	Inicia nome, preco e validade.
Construtor	nome, validade	Inicia nome e validade.
setValidade	dia, mes, ano	Define a validade via parâmetros individuais.
setValidade	data	Define a validade via objeto Data .
vender	qtde	Diminui a quantidade se houver estoque.
comprar	qtde	Aumenta a quantidade em estoque.

Além dos métodos anteriores, classe deve conter métodos **get** e **set** para os atributos nome, preco, quantidade e método **toString()**.

Dica

Utilize o conceito de **sobrecarga** (*overload*).

Classe Principal

Desenvolva uma classe **Principal** contendo um vetor de **Produto**.

- Inclua pelo menos dois produtos, utilizando métodos construtores diferentes para cada um.
- Execute as duas variantes do método **setValidade()**.

3. Imóveis

O objetivo deste exercício é modelar um Imóvel residencial, aplicando conceitos avançados de modelagem estrutural:

Composição: Um Imóvel possui cômodos essenciais (Comodo) que pertencem exclusivamente a ele. Esses cômodos são instanciados estritamente de dentro do construtor do Imóvel e não existem de forma autônoma fora dele.

Agregação: Um Imóvel possui uma lista de móveis ou itens de decoração (Mobilia). Contudo, esses itens podem ser adquiridos e criados de forma independente antes de serem alocados ao imóvel.

Classe mobilia

Representa um móvel que será inserido no imóvel.

Atributos

Identificador	Tipo	Descrição
descricao	String	Descrição detalhada do móvel (ex: "Sofá 3 lugares")
tipo	String	Categoria da mobília (ex: "Sala", "Quarto")

Métodos

Identificador	Retorno	Parâmetro	Descrição
construtor	–	descricao, tipo	Inicializa os atributos da mobília.
getDescricao	String	–	Retorna a descrição do móvel.

Classe Comodo (Parte da Composição)

Representa um cômodo estrutural do imóvel (ex: Sala, Cozinha).

Atributos

Identificador	Tipo	Descrição
nomeComodo	String	Nome do cômodo (ex: "Cozinha", "Quarto Principal")
areaMetros	double	Área do cômodo em metros quadrados

Métodos

Identificador	Retorno	Parâmetro	Descrição
construtor	–	nomeComodo, areaMetros	Inicializa o nome e a área do cômodo.
exibirComodo	void	–	Imprime no console os dados estruturais do cômodo.

Classe Imovel (O Todo)

Representa o imóvel residencial que gerencia os cômodos (composição) e a mobília (agregação).

Atributos

Identificador	Tipo	Descrição
endereco	String	Endereço completo do imóvel
comodos	List	Lista interna de cômodos (Composição)
mobalias	List	Lista externa de mobílias associadas (Agregação)

Métodos

Identificador	Retorno	Parâmetro	Descrição
construtor	–	endereco	Inicializa o endereço, cria a lista de mobalias vazia e instancia internamente os cômodos padrão do imóvel (garantindo a composição).
adicionarMobilia	void	Mobilia m	Adiciona um item de mobília preexistente à lista (Agregação).
exibirDetalhes	void	–	Imprime o endereço, percorre os cômodos internos e lista as mobílias presentes.

Classe principal

desenvolver a classe de teste principal contendo o método main. Nele, é necessário:

- Instanciar objetos da classe Mobilia de forma independente (demonstrando a criação prévia para a agregação).
- Instanciar um objeto da classe Imovel, o qual gerará automaticamente seus cômodos internos por meio do construtor (evidenciando a composição).
- Adicionar as mobílias criadas ao imóvel utilizando o método de agregação correspondente.
- Acionar o método de exibição de detalhes para validar a consolidação de toda a estrutura no console.

6 Mecanismos de abstração

Na programação Orientada a Objetos utilizamos o termo **concreto(a)** para definir classes e métodos que são implementados de forma completa.

Todas as classes e métodos criados até então eram do tipo concretos, ou seja, tanto as super classes como as subclasses implementavam todos os métodos. Além disso, as classes concretas participavam do sistema diretamente.

Observação

Em Java, costumamos chamar as classes concretas de POJO (*Plain Old Java Object*). Teoricamente é possível ter POJO não concreto, mas na prática isso não é comum. Características de um POJO:

- Sem herança forçada: Não estende classes de frameworks (como `HttpServlet` ou `ActionForm`).
- Sem interfaces obrigatórias: Não implementa interfaces complexas de terceiros.
- Sem anotações obrigatórias: Não exige anotações predefinidas para funcionar.
- Simplicidade: Contém apenas atributos, métodos de acesso (getters/setters) e regras de negócio simples.

Entretanto, no desenvolvimento de software, algumas super classes não fazem sentido serem instanciadas, no exemplo da clinica veterinária, não vamos criar um objeto **pessoa** e sim veterinário ou cliente.

Esse tipo de classe pode servir, em alguns casos, apenas de modelos para as subclasses ou fazer parte do **encapsulamento**.

6.1 Classes abstratas

Uma classe abstrata serve como uma base estrutural que não deve ser utilizada de forma isolada. Por representar conceitos muito genéricos, ela permanece incompleta por design. Assim, o papel da classe abstrata é definir um “molde” que precisa ser concretizado pelas subclasses, as quais herdarão e preencherão as lacunas de implementação necessárias para que o objeto possa ser funcional (Leite et al. (2016)).

Essas classe têm uma característica importante: **não podem ser instanciadas**. Ou seja, não podemos criar objetos diretamente a partir delas (Leite et al. (2016)). Logo,

essas classes devem estar no topo da hierarquia de classes.

Observação

Classes abstratas não podem ser instanciadas. Logo elas devem estar no topo da hierarquia de classes.

Para criarmos uma classe abstrata em Java utilizamos a palavra **abstract** antes do **class** e do nome da classe.

```
abstract class Pessoa {
    protected String nome;
    protected String cpf;
    protected String email;
    // Composicao: Pessoa "tem um" Endereco
    private Endereco endereco;

    public Pessoa(String nome, String cpf, String email, String
        rua, String cidade) {
        this.nome = nome;
        this.cpf=cpf;
        this.email=email;
        this.endereco = new Endereco(rua, cidade);
    }
}
```

6.1.1 Métodos abstratos

Podemos também definir métodos como abstratos. A ideia é para que método também sirva de molde. Para isso, **ele não deve possuir uma implementação**, mas sim apenas a definição de sua assinatura (Leite et al. (2016)). Portanto as subclasses devem implementar esses métodos, utilizando a sobreescrita **@Override**.

Observação

- Uma classe abstrata não precisa necessariamente ter um método abstrato.
- Um método abstrato deve ser implementado nas classes filhas utilizando a notação **@Override**.

Veja o exemplo de uso de um método abstrato.

```
abstract class Pessoa {
    protected String nome;
    protected String cpf;
    protected String email;
```

```
// Composicao: Pessoa "tem um" Endereco
private Endereco endereco;

public Pessoa(String nome, String cpf, String email, String
    rua, String cidade) {
    this.nome = nome;
    this.cpf=cpf;
    this.email=email;
    this.endereco = new Endereco(rua, cidade);

// Exemplo de metodo abstrato:
// Define um contrato que toda subclasse deve obrigatoriamente
    seguir,
// mas nao define como ele sera executado aqui na superclasse.
public abstract String getDescricao();
}
```

6.2 Interfaces

Quando um sistema exige uma estrutura mais elaborada, a interface atua como um contrato de comportamento. Ela define um conjunto de métodos que devem obrigatoriamente ser implementados por qualquer classe que a utilize. O ponto central é que a interface foca apenas na definição do que deve ser feito, deixando a liberdade da implementação real para cada classe específica (Leite et al. (2016)).

Em Java a palavra **interface** deve ser utilizada antes da palavra **class** e do nome da interface. Diferente das classes abstratas, uma interface **não pode conter atributos de instância**, apenas constantes (variáveis estáticas e finais) e métodos abstratos. Uma classe que implementa uma interface deve utilizar a palavra-chave **implements** e fornecer implementações concretas para todos os métodos definidos na interface.

Vamos adionar uma interface chamada Tratamento em nosso exemplo da clínica veterinária (para fins didáticos). Essa interface terá um método chamado tratarAnimal() que será implementado por diferentes classes de tratamento, como TratamentoVeterinario e TratamentoEstetico.

```
public interface Tratamento {
    void tratarAnimal(Animal animal);
}

public class TratamentoVeterinario implements Tratamento {
    @Override
    public void tratarAnimal(Animal animal) {
        // Implementacao do tratamento veterinario
        System.out.println("Tratando o animal " + animal.getNome()
            + " com cuidados veterinarios.");
    }
}
```

```

public class TratamentoEstetico implements Tratamento {
    @Override
    public void tratarAnimal(Animal animal) {
        // Implementacao do tratamento estetico
        System.out.println("Tratando o animal " + animal.getNome()
            + " com cuidados esteticos.");
    }
}

```

6.3 Métodos estáticos e default

Os métodos estáticos e default foram introduzidos em interfaces a partir do Java 8, permitindo que as interfaces tenham implementações de métodos. A implementação de métodos estáticos pode ser observado na seção 4.5 do capítulo anterior.

Logo, com a chegada dos métodos default e static, as interfaces ganharam recursos que aproximam muito a sua utilização prática da de uma classe abstrata, permitindo o compartilhamento de código pronto. No entanto, elas continuam tendo diferenças arquiteturais fundamentais que definem quando usar cada uma.

Observação

Nas interfaces, os métodos são implicitamente públicos e abstratos, enquanto nas classes abstratas, os métodos podem ter diferentes níveis de acesso e podem ser concretos ou abstratos.

6.3.1 Herança múltipla

A herança múltipla não é suportada em Java para classes, mas é possível implementar múltiplas interfaces em uma única classe. Isso permite que uma classe herde o comportamento de várias interfaces.

Um exemplo prático seria uma classe que implementa tanto a interface `Tratamento` quanto a interface `Autenticavel`, permitindo que ela trate animais e também gerencie autenticação de usuários.

6.4 Exemplo

Para integrar os conceitos de orientação a objetos na estrutura do diagrama, podemos transformar a superclasse `Pessoa` em uma classe abstrata (forçando um método de exibição) e introduzir uma interface `Autenticavel` para definir regras de acesso ou credenciais ao sistema.

```

interface Autenticavel {
    // Constante implícita
    int NIVEL_ACESSO_PADRAO = 1;
}

```

```
boolean autenticar(String credencial);

// Método default (Java 8+)
default void exibirCabecalhoSeguranca() {
    System.out.println("[Sistema Veterinário - Acesso Restrito
                        ]");
}
}
```

```
class Endereco {
    private String rua;
    private String cidade;

    public Endereco(String rua, String cidade) {
        this.rua = rua;
        this.cidade = cidade;
    }

    @Override
    public String toString() {
        return rua + ", " + cidade;
    }
}
```

```
abstract class Pessoa {
    protected String nome;
    protected String cpf;
    protected String email;
    protected Endereco endereco; // Composição (1 para 1)

    public Pessoa(String nome, String cpf, String email, String
        rua, String cidade) {
        this.nome = nome;
        this.cpf = cpf;
        this.email = email;
        this.endereco = new Endereco(rua, cidade); // Composição
        estrita
    }

    // Método abstrato: obriga as subclasses a implementarem a
    // exibição de dados
    public abstract void exibirInformacoes();

    public String getNome() {
        return nome;
    }
}
```

6 Mecanismos de abstração

```
class Veterinario extends Pessoa implements Autenticavel {
    private String crmv;

    public Veterinario(String nome, String cpf, String email,
        String rua, String cidade, String crmv) {
        super(nome, cpf, email, rua, cidade);
        this.crmv = crmv;
    }

    @Override
    public void exibirInformacoes() {
        System.out.println("Veterinário: " + nome + " | CRMV: " +
            crmv + " | Endereço: " + endereco);
    }

    @Override
    public boolean autenticar(String credencial) {
        // Validação simples simulada baseada no CRMV
        return this.crmv.equals(credencial);
    }

    public void realizarConsulta(Consulta consulta) {
        System.out.println("O veterinário " + nome + " está
            realizando a consulta.");
    }
}
```

```
class Pet {
    private String nomeAnimal;
    private int idadeAnimal;
    private int sexo; // 0 para Femea, 1 para Macho

    public Pet(String nomeAnimal, int idadeAnimal, int sexo) {
        this.nomeAnimal = nomeAnimal;
        this.idadeAnimal = idadeAnimal;
        this.sexo = sexo;
    }

    public String getNomeAnimal() {
        return nomeAnimal;
    }
}
```

```
class Consulta {
    private String data;
    private List<Pet> petsAtendidos;
    private Veterinario veterinario; // Associação

    public Consulta(String data, Veterinario veterinario) {
```

6 Mecanismos de abstração

```
        this.data = data;
        this.veterinario = veterinario;
        this.petsAtendidos = new ArrayList<>();
    }

    public void adicionarPet(Pet pet) {
        this.petsAtendidos.add(pet);
    }

    public void finalizarConsulta() {
        System.out.println("Consulta na data: " + data);
        veterinario.exibirInformacoes();
        System.out.println("Pets atendidos nesta sessão:");
        for (Pet p : petsAtendidos) {
            System.out.println(" - " + p.getNomeAnimal());
        }
    }
}
```

```
public class Principal {
    public static void main(String[] args) {
        // Instanciando o Veterinário (Pessoa abstrata +
        // Autenticavel)
        Veterinario vet = new Veterinario("Dr. Carlos", "
            123.456.789-00", "carlos@vet.com", "Rua das Flores, 100
            ", "Ponta Grossa", "CRMV-PR 9876");

        // Testando métodos da interface e da classe abstrata
        vet.exibirCabecalhoSeguranca();
        if (vet.autenticar("CRMV-PR 9876")) {
            System.out.println("Acesso autorizado para o veteriná
                rio!");
        }

        // Criando Pets (Agregação)
        Pet pet1 = new Pet("Mel", 3, 0);
        Pet pet2 = new Pet("Thor", 5, 1);

        // Criando a Consulta e associando o veterinário
        Consulta consulta = new Consulta("03/09/2026", vet);
        consulta.adicionarPet(pet1);
        consulta.adicionarPet(pet2);

        consulta.finalizarConsulta();
    }
}
```

O diagrama abaixo ilustra a relação entre as classes e a interface, destacando a herança, implementação e associações.

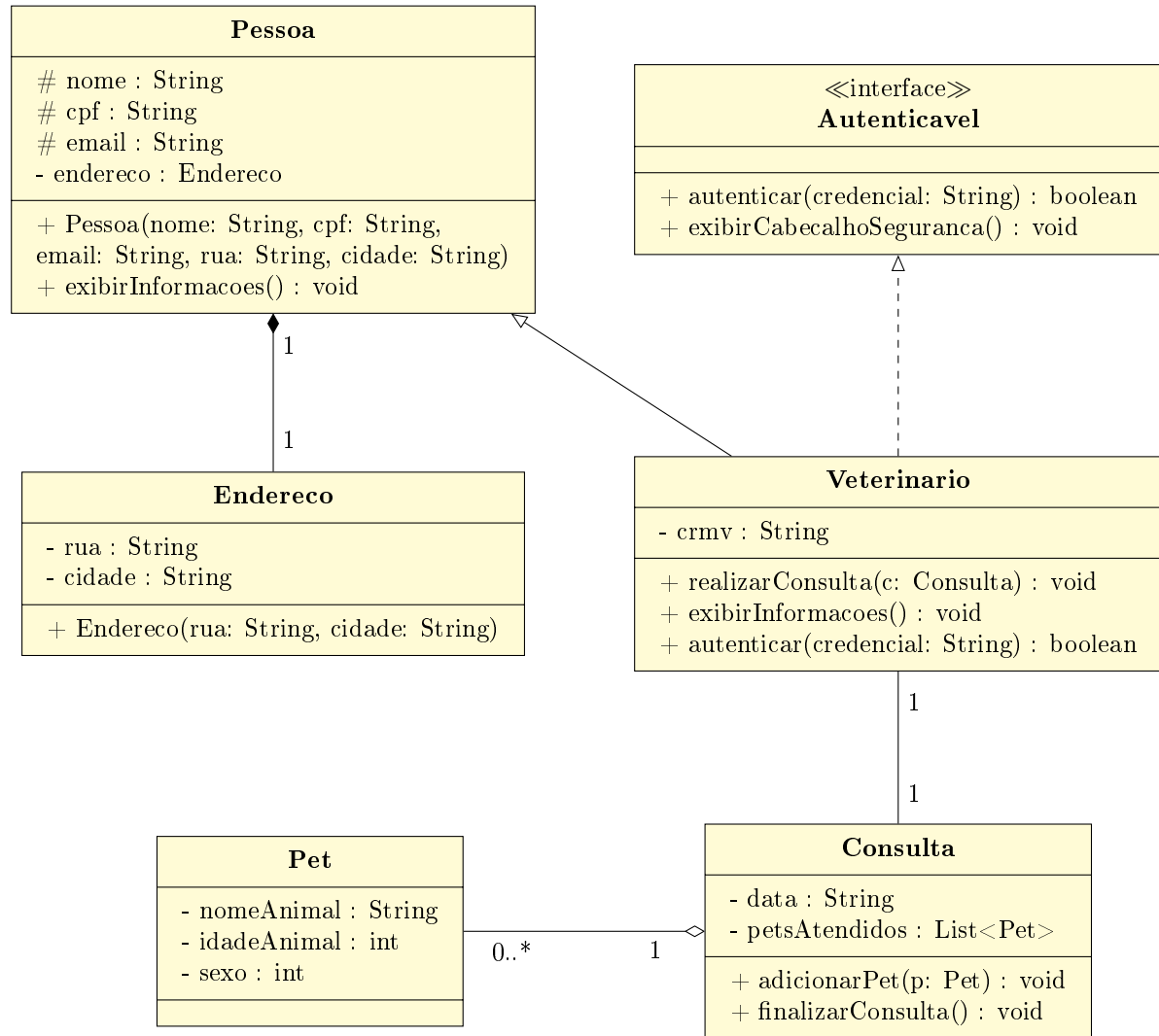


Figura 6.1: Diagrama UML atualizado incorporando a interface **Autenticavel** e a classe abstrata **Pessoa**.

6.5 Comparação entre classes abstratas e interfaces

Abaixo está uma tabela comparativa entre classes abstratas e interfaces, destacando suas principais diferenças e usos:

Característica	Classe Abstrata	Interface
Instanciação	Não pode ser instanciada	Não pode ser instanciada
Atributos	Pode ter atributos de instância	Apenas constantes (static final)
Métodos	Pode ter métodos concretos e abstratos	Apenas métodos abstratos (Java 8+ permite métodos default e static)
Herança	Pode estender apenas uma classe abstrata	Pode implementar múltiplas interfaces
Construtores	Pode ter construtores	Não pode ter construtores

Tabela 6.1: Comparação entre Classes Abstratas e Interfaces

Exercícios**1. Exercícios teóricos**

Suponha que um sistema possui uma **A** com os métodos abstratos públicos `metodo1()` e `metodo2()`. A classe **B** é uma classe abstrata, implementa a classe **A** e possui dois métodos, o `metodo3()` concreto e `metodo4()` abstrato. Por último, a classe **C** é uma herança de **B**.

- O que acontece quando classe **B** não implementa todos os métodos da interface? Justifique sua resposta.
- A classe **C** é obrigada a implementar todos os métodos da interface? Justifique sua resposta.
- A classe **C** é obrigada a implementar todos os métodos da classe **B**? Justifique sua resposta.
- Se o método `metodo3()` for do tipo final na classe **B**, a classe **C** poderá sobrescrevê-lo? Justifique sua resposta.

2. Sistema de Gerenciamento de Livros

O objetivo deste exercício é consolidar o uso de **interfaces** para definir capacidades específicas e **classes abstratas** para estabelecer identidades comuns em um sistema de biblioteca e livraria.

Interface ItemDeBiblioteca

Crie a interface `ItemDeBiblioteca` contendo os seguintes métodos contratuais de empréstimo:

Identificador	Retorno	Parâmetro	Descrição
<code>emprestarItem</code>	<code>void</code>	–	Realiza o empréstimo do item, alterando seu estado interno.
<code>devolverItem</code>	<code>void</code>	–	Registra a devolução do item ao acervo.
<code>verificarSeEstaEmprestado</code>	<code>boolean</code>	–	Retorna verdadeiro se o item estiver emprestado no momento.

Classe Abstrata Livro

Desenvolva a classe abstrata **Livro** com os atributos fundamentais e métodos compartilhados: * **Atributos privados/protegidos:** **nome** (String), **autor** (String), **ano** (int), **avaliacao** (int, variando de 0 a 10).

Identificador	Retorno	Parâmetro	Descrição
construtor	–	nome, autor, ano	Inicializa os dados básicos do livro e define a avaliação inicial como zero.
toString	String	–	Retorna uma representação formatada em texto com os dados do livro.
avaliar	void	–	Utiliza a classe Scanner para solicitar e armazenar a nota do livro dada pelo usuário (0 a 10).
informar	void (abstract)	–	Método abstrato que força as subclasses a definirem o comportamento de indicação de origem.

Classes Concretas

Implemente as especializações do sistema: * **LivroDeBiblioteca:** Estende **Livro** e implementa a interface **ItemDeBiblioteca**. Adicione atributos de controle de empréstimo (ex: **boolean emprestado**), construtor, reescrita de **toString()**, o método abstrato **informar()** e a lógica completa dos métodos da interface. * **LivroDeLivraria:** Estende **Livro**. Adicione o atributo **preco** (double), construtor, reescrita de **toString()**, implementação do método **informar()** e crie o método **imprimirPreco()**.

Classe Principal

O aluno deve desenvolver a classe **Principal** contendo o método ‘main’. Nele, é necessário: * Criar um vetor (ou lista) do tipo genérico **Livro** contendo instâncias mistas de **LivroDeBiblioteca** e **LivroDeLivraria**. * Utilizar um laço de repetição para percorrer todos os elementos da estrutura: * Imprimir as informações gerais chamando o **toString()**. * Executar os métodos **informar()** e **avaliar()** polimorficamente. * Utilizar a verificação de tipo (**instanceof**) para executar o método **imprimirPreco()** caso seja

um livro de livraria, ou gerenciar o fluxo de empréstimo (`emprestarItem()`) caso seja um livro de biblioteca.

Dica

Note como o uso da interface `ItemDeBiblioteca` permite que apenas a classe `LivroDeBiblioteca` possua a capacidade de empréstimo, mantendo o ecossistema limpo sem poluir a classe `LivroDeLivraria` com regras que não pertencem ao seu modelo de negócio.

Exercícios

1. **Exercícios teóricos**

Suponha que um sistema possui uma `A` com os métodos abstratos públicos `metodo1()` e `metodo2()`. A classe `B` é uma classe abstrata, implementa a classe `A` e possui dois métodos, o `metodo3()` concreto e `metodo4()` abstrato. Por último, a classe `C` é uma herança de `B`.

- O que acontece quando classe `B` não implementa todos os métodos da interface? Justifique sua resposta.
- A classe `C` é obrigada a implementar todos os métodos da interface? Justifique sua resposta.
- A classe `C` é obrigada a implementar todos os métodos da classe `B`? Justifique sua resposta.
- Se o método `metodo3()` for do tipo final na classe `B`, a classe `C` poderá sobrescrevê-lo? Justifique sua resposta.

2. **Sistema de Gerenciamento de Livros**

O objetivo deste exercício é consolidar o uso de **interfaces** para definir capacidades específicas e **classes abstratas** para estabelecer identidades comuns em um sistema de biblioteca e livraria.

Interface `ItemDeBiblioteca`

Crie a interface `ItemDeBiblioteca` contendo os seguintes métodos contratuais de empréstimo:

Identificador	Retorno	Parâmetro	Descrição
emprestarItem	void	–	Realiza o empréstimo do item, alterando seu estado interno.
devolverItem	void	–	Registra a devolução do item ao acervo.
verificarSeEstaEmprestado	boolean	–	Retorna verdadeiro se o item estiver emprestado no momento.

Classe Abstrata Livro

Desenvolva a classe abstrata **Livro** com os atributos fundamentais e métodos compartilhados: * **Atributos privados/protegidos:** **nome** (String), **autor** (String), **ano** (int), **avaliacao** (int, variando de 0 a 10).

Identificador	Retorno	Parâmetro	Descrição
construtor	–	nome, autor, ano	Inicializa os dados básicos do livro e define a avaliação inicial como zero.
toString	String	–	Retorna uma representação formatada em texto com os dados do livro.
avaliar	void	–	Utiliza a classe Scanner para solicitar e armazenar a nota do livro dada pelo usuário (0 a 10).
informar	void (abstract)	–	Método abstrato que força as subclasses a definirem o comportamento de indicação de origem.

Classes Concretas

Implemente as especializações do sistema: * **LivroDeBiblioteca:** Estende **Livro** e implementa a interface **ItemDeBiblioteca**. Adicione atributos de controle de empréstimo (ex: **boolean emprestado**), construtor, reescrita de **toString()**, o método abstrato **informar()** e a lógica completa dos métodos da interface. * **LivroDeLivraria:** Estende **Livro**. Adicione o atributo **preco** (double), construtor, reescrita de **toString()**, implementação do método **informar()** e crie o método **imprimirPreco()**.

Classe Principal

O aluno deve desenvolver a classe `Principal` contendo o método `main`. Nele, é necessário: * Criar um vetor (ou lista) do tipo genérico `Livro` contendo instâncias mistas de `LivroDeBiblioteca` e `LivroDeLivraria`. * Utilizar um laço de repetição para percorrer todos os elementos da estrutura: * Imprimir as informações gerais chamando o `toString()`. * Executar os métodos `informar()` e `avaliar()` polimorficamente. * Utilizar a verificação de tipo (`instanceof`) para executar o método `imprimirPreco()` caso seja um livro de livraria, ou gerenciar o fluxo de empréstimo (`emprestarItem()`) caso seja um livro de biblioteca.

Dica

Note como o uso da interface `ItemDeBiblioteca` permite que apenas a classe `LivroDeBiblioteca` possua a capacidade de empréstimo, mantendo o ecossistema limpo sem poluir a classe `LivroDeLivraria` com regras que não pertencem ao seu modelo de negócio.

Referências Bibliográficas

Cabral, L. P. (2014). Java e orientação a objetos.

Deitel, H. M. and Deitel, P. J. (2015). *Como programar em Java*. Pearson educacion.

Leite, T. et al. (2016). *Orientação a Objetos: Aprenda seus conceitos e suas aplicabilidades de forma efetiva*. Editora Casa do Código.